AI FOR IT PROFESSIONALS

Prompts for IT Professionals

Expanded field guide to prompt engineering for systems, cloud, security, automation, and support.

EDITION 2.0

16 DISCIPLINES | 48 PRACTICAL TEMPLATES | WORKED EXAMPLES

THE FIELD PRINCIPLE

Give the model the facts, ask it to expose uncertainty, test the safest explanation, and keep the engineering decision with the operator.

Rhyan Jack
Author and Editor

READER GUIDANCE

Copyright, responsible use, and how to work with this guide

AI DEVELOPMENT DISCLOSURE

Generative AI (GPT Sol 5.6) was used in developing portions of the text and visual content. The author selected, reviewed, edited, structured, and designed the final publication. Model output remains fallible and must not be treated as an operational approval, security verdict, legal conclusion, or substitute for engineering judgement.

HOW TO USE THIS BOOK

1 Choose	2 Replace	3 Review	4 Refine
Start with the discipline and task that match the work.	Insert your environment, scope, evidence, changes, and constraints.	Check assumptions, safety, compatibility, and authority.	Feed back results and ask the model to update its reasoning.

CORE PRINCIPLE

Use AI to accelerate investigation, comparison, drafting, and review - not to replace evidence, policy, change authority, or professional judgement. The final decision belongs to the authorised operator who understands the live environment and accepts the risk.

PROTECT YOUR DATA

- Use an approved AI service and follow data-classification policy
- Remove passwords, tokens, private keys, personal data, and customer content
- Prefer short sanitised evidence extracts over complete logs or files

PROTECT THE ENVIRONMENT

- Start read-only and define the permitted authority
- Test on a small known scope before broad execution
- Require validation, abort, rollback, and an accountable owner

CONTENTS

Sixteen practical disciplines, each built as context, worked example, and prompt library

THE THREE-PAGE CHAPTER SYSTEM

PAGE 1 - Understand the discipline and the evidence the model needs. PAGE 2 - Study a worked prompt, its response shape, and the next-turn refinements. PAGE 3 - Copy three focused templates and run the pre-submit check.

A NOTE ON COPY-READY PROMPTS

Bracketed fields are instructions to you, not content for the model. Replace them, remove irrelevant clauses, and state unknown where a fact is not yet available. Never fabricate a value simply to make the template look complete.

FIELD FOUNDATION

Prompt engineering for real IT work

A compact operating model for every chapter that follows

THE PRACTICAL DEFINITION

Prompt engineering in IT is disciplined problem framing. It is the work of giving a model enough verified context to support a technical decision, constraining what it may assume or recommend, and asking for an answer that can be checked by an operator.

A longer prompt is not automatically better. Include the details that change diagnosis, risk, or action. Remove narrative that does not. When the situation evolves, provide a fresh fact pack and ask the model to update its reasoning.

USE A RESPONSE CONTRACT

ASK FOR STRUCTURE

Useful outputs include ranked hypotheses, evidence for and against, discriminating tests, result branches, owners, validation, stop conditions, rollback, and a concise summary for the next operator.

DO NOT OUTSOURCE THE DECISION

The model can organise evidence and challenge a plan. It cannot see every dependency, verify authority, or accept operational risk.

THE FIELD SEQUENCE

01	02	03	04	05
Role and standard	**Environment and scope**	**Evidence and change**	**Constraints and authority**	**Output and review**

UNIVERSAL STARTING PROMPT

```
Act as my senior [discipline] engineer. Environment and service: [details]. Symptom, onset, and scope: [details]. Confirmed evidence with timestamps: [details]. Healthy comparison: [details]. Recent changes: [details]. Constraints and authority: [details]. Ask only the questions that would materially change the plan. Then separate facts, assumptions, and unknowns; rank hypotheses; give read-only discriminating tests with interpretation; and define validation, stop, escalation, and rollback conditions before any change.
```

ITERATE WITH EVIDENCE

- Return exact results with target and timestamp
- Ask what the result proves and does not prove
- Require the hypothesis ranking to change when evidence changes
- Start a fresh summary when the thread becomes long or confused

REVIEW OUTSIDE THE MODEL

- Check primary documentation and version compatibility
- Validate in a lab or small known scope
- Use peer review for high-impact or unfamiliar change
- Keep the ticket, change record, or incident log authoritative

THE STANDARD

Specific enough to reason. Constrained enough to be safe. Structured enough to review.

CHAPTER 01

The Senior Engineer Prompt

Frame the problem so the model can reason like a careful technical peer

DISCIPLINE CONTEXT

A senior-engineer prompt is not role-play for its own sake. It is a compact technical brief. The role sets the expected level of judgement, but the useful work comes from the context, evidence, constraints, and requested output that follow it.

The model should know what system is involved, what changed, what is affected, what still works, and what you have already observed. It should also know which actions are unavailable. A clear response contract then turns a broad answer into ranked causes, read-only checks, interpretation, and a decision point.

PROMPT ENGINEERING LENS

WHY IT MATTERS

Treat the prompt as a handover to an experienced colleague arriving without local knowledge. Separate confirmed facts from assumptions and unknowns. Ask the model to say when the evidence is too weak to justify a recommendation.

FIELD RULE

Role sets the standard. Evidence sets the direction. Constraints keep the advice usable.

THE DISCIPLINE SEQUENCE

01	02	03	04	05
Role	**Context**	**Evidence**	**Constraints**	**Output**

INCLUDE IN THE PROMPT

- System, service, version, and business purpose
- Exact symptom, onset, scope, and healthy comparisons
- Commands, events, metrics, and recent changes already checked
- Change window, access, policy, and availability constraints

AVOID THESE WEAK PATTERNS

- A job title with no environment detail
- Vague requests such as fix this or what is wrong
- Unlabelled assumptions presented as facts
- Requests for commands without validation or rollback

MAKE THE REASONING VISIBLE

EVIDENCE	FALSIFY	CONTROL
What fact would most change the current ranking?	What result would disprove the leading cause?	Which next step is safe, reversible, and authorised?

OPERATIONAL CONTEXT

This pattern is most valuable when an issue crosses teams or experience levels. A consistent brief lets a service-desk analyst, infrastructure engineer, or incident lead see the same scope, evidence, and constraints. It also creates a cleaner handoff because the model's answer can be reviewed against an explicit request rather than an implied conversation.

CHAPTER 01

The Senior Engineer Prompt

Frame the problem so the model can reason like a careful technical peer

WORKED FIELD EXAMPLE

SCENARIO

Remote staff report slow access to one Windows file share after a security policy change. Server CPU and storage latency are normal. Authentication events appear several seconds late. No restart is allowed during business hours.

COPY-READY PROMPT

```
Act as my senior systems engineer. Environment: Windows Server 2022 file server used by remote staff. Symptom: one share became slow after a security policy change. Scope: remote users are affected; a local test account is normal. Evidence: CPU and storage latency are normal, but authentication events are delayed. Constraint: read-only checks only and no restart during business hours. Give me: (1) ranked causes, (2) the safest discriminating check for each, (3) what each result would mean, and (4) the point at which you would escalate or change direction. Mark assumptions and unknowns.
```

WHY THIS PROMPT WORKS

● Names the environment and a working comparison	● Places the recent change beside objective evidence
● Defines the allowed level of risk	● Requests interpretation, not a command dump

CONDENSED SENIOR ENGINEER RESPONSE

Start by locating the delay in the user journey: name resolution, authentication, authorization, or file-service response. Compare the remote user with the local test using timestamped checks. Review policy processing and authentication events around the same request, then confirm the remote path and domain-controller selection. If authentication is delayed before the file service receives the request, focus on identity, DNS, or policy. If authentication is timely but file access is slow, move to share, endpoint security, and transport evidence. Do not restart a service until the failing layer is identified and a success measure is defined.

NEXT-TURN REFINEMENT 1

```
Ask me up to five questions that would most change your ranking before you recommend a fix.
```

NEXT-TURN REFINEMENT 2

```
Critique your plan for hidden assumptions, unsafe steps, and evidence that could disprove the leading cause.
```

WHAT THIS EXAMPLE DOES NOT PROVE

The senior-engineer label does not give the model private knowledge of your estate, confirm that a command matches your build, or make a recommendation authorised. The result remains only as strong as the supplied facts and the operator's independent checks.

TURN THE ANSWER INTO EVIDENCE

01	02	03	04	05
Confirm facts	**Run one safe check**	**Interpret result**	**Update the plan**	**Record the decision**

CHAPTER 01

The Senior Engineer Prompt

Frame the problem so the model can reason like a careful technical peer

PROMPT LIBRARY

Use these as starting structures. Replace every bracketed field, remove anything irrelevant, and keep unknowns explicit. The strongest template is the one that matches the real decision you need to make.

STRUCTURED DIAGNOSTIC BRIEF

USE WHEN: you need a dependable first answer

```
Act as my senior [discipline] engineer. Environment: [platform, version, role, location]. Business service: [purpose]. Symptom and onset: [exact behaviour and time]. Scope: [affected and unaffected users, devices, sites, or instances]. Evidence already collected: [facts with timestamps]. Recent changes: [known changes or none]. Constraints: [read-only, maintenance window, permissions, policy]. Rank the likely causes, give the lowest-risk check that separates each cause, explain every result, and state assumptions, unknowns, stop conditions, and escalation triggers.
```

EVIDENCE-GAP INTERVIEW

USE WHEN: your first report is incomplete

```
Act as a senior engineer receiving an incomplete incident handover. Do not diagnose yet. Review these facts: [paste sanitised facts]. Ask no more than seven questions, ordered by how much each answer would change scope, urgency, or the leading hypothesis. For each question, explain why it matters and what answer patterns would send the investigation in different directions. Do not request passwords, secrets, personal data, or unnecessary raw logs.
```

TECHNICAL PEER REVIEW

USE WHEN: you already have a proposed solution

```
Review this proposed diagnosis and fix as a sceptical senior engineer: [proposal]. Environment and constraints: [details]. Separate confirmed facts, inferences, and unsupported claims. Identify the blast radius, prerequisites, failure modes, validation steps, rollback conditions, and missing approvals. Then provide a safer revised recommendation. If evidence is insufficient, say what must be confirmed before any change.
```

BEFORE YOU SUBMIT

- ✓ Facts and assumptions are labelled
- ✓ Risk constraints are explicit
- ✓ No secrets or unnecessary sensitive data
- ✓ Scope includes a healthy comparison
- ✓ Output shape supports a decision

TEMPLATE USE SEQUENCE

01	02	03	04	05
Replace fields	**Remove irrelevant text**	**Mark unknowns**	**Check data handling**	**Verify the output**

CHAPTER 02

Troubleshooting as a Reasoning Loop

Move from observation to hypothesis, test, validation, and deliberate revision

DISCIPLINE CONTEXT

Good troubleshooting is an evidence loop, not a list of favourite commands. A useful prompt asks the model to define scope, group possible causes into technical domains, and choose tests that separate those domains with the least risk and effort.

The important step is interpretation. Every check should have a reason, an expected signal, and a next branch. When evidence weakens a hypothesis, instruct the model to revise the ranking rather than defend its first answer. This keeps the conversation diagnostic instead of turning it into a rigid script.

PROMPT ENGINEERING LENS

WHY IT MATTERS

Ask for discriminating tests: checks that produce meaningfully different results for competing explanations. Feed each result back into the model with the original facts so it can update the plan without losing context.

FIELD RULE

The next check should reduce uncertainty, not simply produce more data.

THE DISCIPLINE SEQUENCE

01	02	03	04	05
Observe	**Scope**	**Hypothesise**	**Test**	**Validate**

INCLUDE IN THE PROMPT

- Affected versus healthy paths
- Time of onset and last known good state
- Recent changes and already failed attempts
- Desired stopping point or escalation boundary

AVOID THESE WEAK PATTERNS

- Testing several variables at once
- Continuing after evidence contradicts the plan
- Treating correlation as proof
- Applying a fix before defining success

MAKE THE REASONING VISIBLE

EVIDENCE	FALSIFY	CONTROL
What fact would most change the current ranking?	What result would disprove the leading cause?	Which next step is safe, reversible, and authorised?

OPERATIONAL CONTEXT

Run the loop in short rounds. Record the hypothesis, test, target, timestamp, result, and interpretation before moving on. This makes backtracking possible and prevents the same check being repeated by another engineer. The model can maintain the reasoning structure, while the ticket or incident record remains the source of truth.

CHAPTER 02

Troubleshooting as a Reasoning Loop

Move from observation to hypothesis, test, validation, and deliberate revision

WORKED FIELD EXAMPLE

SCENARIO

A business application fails for users in one office. Other offices are healthy. The application team reports no service-wide fault. The affected site can reach other internal services and no change has yet been confirmed.

COPY-READY PROMPT

```
Act as my senior troubleshooting partner. A business application fails for users in Office A only; Offices B and C are healthy. Other internal services work from Office A. No service-wide alert exists. Build an evidence-led reasoning loop. First define whether the fault is most likely tied to site, user, device, network path, identity, or application edge. Rank the domains, then give one low-risk discriminating test per domain. For every test, state the hypothesis, expected positive and negative result, and the next branch. Stop after the first five checks and ask me for the results before continuing.
```

WHY THIS PROMPT WORKS

- Prevents an uncontrolled troubleshooting tree
- Uses unaffected offices as a baseline
- Makes each test answer a stated question
- Forces the model to pause and reassess

CONDENSED SENIOR ENGINEER RESPONSE

First prove the scope with one affected user and one known-good user on the same device or network where possible. Then compare name resolution, route, authentication, and application reachability between Office A and a healthy office. A site-specific DNS or policy difference should produce a consistent result across users; a user-specific identity issue should follow the account; a device issue should follow the endpoint. Record timestamps and do not change configuration during the comparison. After five checks, update the domain ranking and choose the next test from the remaining uncertainty rather than from the original guess.

NEXT-TURN REFINEMENT 1

```
Here are the five results. Update the hypothesis ranking and explain exactly what changed.
```

NEXT-TURN REFINEMENT 2

```
Design the single safest test that would distinguish the top two remaining causes.
```

WHAT THIS EXAMPLE DOES NOT PROVE

A working comparison narrows scope but may hide a second difference. A single failed or successful test can also be misleading if timing, cache, load balancing, or intermittent behaviour changes between attempts. Repeat only when repetition answers a specific uncertainty.

TURN THE ANSWER INTO EVIDENCE

01	02	03	04	05
Confirm facts	**Run one safe check**	**Interpret result**	**Update the plan**	**Record the decision**

CHAPTER 02

Troubleshooting as a Reasoning Loop

Move from observation to hypothesis, test, validation, and deliberate revision

PROMPT LIBRARY

Use these as starting structures. Replace every bracketed field, remove anything irrelevant, and keep unknowns explicit. The strongest template is the one that matches the real decision you need to make.

SCOPE MAPPER

USE WHEN: you do not yet know where the fault lives

```
Act as a senior support engineer. Symptom: [detail]. First known occurrence: [time]. Affected: [users, devices, sites, services]. Unaffected: [healthy comparisons]. Last known good: [time/state]. Recent changes: [details]. Build a scope matrix across user, device, location, network, identity, service, and time. Identify the smallest shared boundary that explains the evidence. Ask only the questions needed to confirm or reject that boundary before proposing remediation.
```

HYPOTHESIS TEST TABLE

USE WHEN: several causes remain plausible

```
Using these confirmed facts [facts], create a ranked hypothesis set. For each hypothesis provide: supporting evidence, contradicting evidence, confidence as low/medium/high, the lowest-risk discriminating test, expected result if true, expected result if false, and the next action for both outcomes. Prefer read-only tests. Do not use a percentage unless the evidence genuinely supports one.
```

ITERATIVE RESULT UPDATE

USE WHEN: you are feeding back test results

```
Continue the same troubleshooting loop. Original facts: [facts]. Previous ranking: [ranking]. New test and result: [result, timestamp, target]. Explain what the result proves, what it does not prove, and how it changes the ranking. Remove causes that are genuinely contradicted, retain unresolved alternatives, and propose only the next two highest-value checks.
```

BEFORE YOU SUBMIT

- ✓ A healthy comparison is available
- ✓ Results have true and false branches
- ✓ Success and stop conditions are defined
- ✓ Each test maps to a hypothesis
- ✓ The plan pauses for reassessment

TEMPLATE USE SEQUENCE

01	02	03	04	05
Replace fields	**Remove irrelevant text**	**Mark unknowns**	**Check data handling**	**Verify the output**

CHAPTER 03

Incident Triage

Use AI to organise fast, safe decisions without losing control of the incident

DISCIPLINE CONTEXT

During an incident, the model should reduce cognitive load, not create a long essay. Give it a timestamped fact set, current business impact, affected scope, known changes, and the people already engaged. Ask for a short triage plan with owners, parallel workstreams, and escalation triggers.

Keep diagnosis and command separate. The first prompts should prioritise containment where necessary, read-only health signals, evidence preservation, and communications. Confidence must remain visible: confirmed facts, likely explanations, and unknowns should never be blended into one confident narrative.

PROMPT ENGINEERING LENS

WHY IT MATTERS

Use the model as an incident scribe and decision-support layer. Refresh it with a compact fact pack at each checkpoint. Never rely on its memory of a long, changing thread for the authoritative incident record.

FIELD RULE

In triage, a clear owner and a safe next decision are more valuable than a perfect theory.

THE DISCIPLINE SEQUENCE

01	02	03	04	05
Impact	**Scope**	**Stabilise**	**Investigate**	**Escalate**

INCLUDE IN THE PROMPT

- Start time, severity, and business effect
- Services, locations, users, and dependencies affected
- Owners, actions in progress, and last update
- Escalation, communication, and change-authority rules

AVOID THESE WEAK PATTERNS

- Speculative root-cause language
- Unapproved remediation during triage
- Unowned action lists
- Invented ETAs or certainty

MAKE THE REASONING VISIBLE

EVIDENCE	FALSIFY	CONTROL
What fact would most change the current ranking?	What result would disprove the leading cause?	Which next step is safe, reversible, and authorised?

OPERATIONAL CONTEXT

Use a fresh, compact fact pack at each incident checkpoint. Include only current impact, evidence, actions, owners, blockers, decisions, and the next update time. This keeps the model aligned with the live event and makes it useful for shift handover, executive summaries, vendor escalation, and action tracking without treating the chat as the incident log.

CHAPTER 03

Incident Triage

Use AI to organise fast, safe decisions without losing control of the incident

WORKED FIELD EXAMPLE

SCENARIO

Internal email is delayed for many users. Transport queues are growing, but inbound and outbound routes may not be equally affected. The incident began 20 minutes ago and a connector change occurred earlier in the day.

COPY-READY PROMPT

```
Act as incident-commander support. Time now: 14:20 UTC. Incident start: approximately 14:00 UTC. Impact: internal mail is delayed for many users; transport queues are growing. Unknown: whether external routes are affected. Recent change: a connector configuration changed at 10:30 UTC. Build a 15-minute triage plan with: scope checks, three parallel workstreams, named role owners rather than people, read-only evidence to collect, communication checkpoints, and explicit triggers for severity increase, rollback review, vendor escalation, or service protection. Separate confirmed facts, hypotheses, and unknowns. Keep the answer to one operational page.
```

WHY THIS PROMPT WORKS

- Anchors the plan to incident time
- Creates parallel work without duplication
- Treats the connector change as evidence, not proof
- Makes escalation conditions observable

CONDENSED SENIOR ENGINEER RESPONSE

Confirm mail-flow scope by route and tenant boundary while one owner reviews transport queues and another checks infrastructure dependencies, provider health, DNS, storage, and connector status. A third owner maintains the fact log and prepares communications. Correlate queue growth with the change timeline, but do not assume causation. Escalate severity if impact expands, delivery latency breaches the agreed threshold, queues continue to grow, or a dependency becomes unstable. Review rollback only after confirming the changed connector is in the failing path and the rollback itself is authorised and lower risk.

NEXT-TURN REFINEMENT 1

```
Convert the latest facts into a five-line incident update without inventing a root cause or ETA.
```

NEXT-TURN REFINEMENT 2

```
At the next checkpoint, show decisions due, owners, blockers, and evidence needed to close each workstream.
```

WHAT THIS EXAMPLE DOES NOT PROVE

The model cannot observe business impact, provider state, or responder progress unless you supply it. A tidy triage plan can still be wrong, late, or unowned. The incident lead must confirm priority, authority, communications, and the acceptable balance between evidence preservation and service recovery.

TURN THE ANSWER INTO EVIDENCE

01	02	03	04	05
Confirm facts	**Run one safe check**	**Interpret result**	**Update the plan**	**Record the decision**

CHAPTER 03

3 / 3

Incident Triage

Use AI to organise fast, safe decisions without losing control of the incident

PROMPT LIBRARY

Use these as starting structures. Replace every bracketed field, remove anything irrelevant, and keep unknowns explicit. The strongest template is the one that matches the real decision you need to make.

FIRST 15 MINUTES

USE WHEN: a live service disruption has just been declared

```
Act as incident-commander support. Current time/timezone: [time]. Start time:
[time/unknown]. Business impact: [impact]. Scope confirmed: [facts]. Unknowns: [unknowns].
Recent changes: [changes]. People/roles engaged: [roles]. Produce a 15-minute plan with
stabilisation, read-only scope checks, parallel workstreams, owners, evidence outputs,
communication time, and escalation triggers. Label facts, hypotheses, and assumptions. Do
not propose an unapproved change.
```

CHECKPOINT DECISION BRIEF

USE WHEN: the incident team needs to reassess

```
Using this timestamped incident fact pack [facts], prepare the next checkpoint. Show:
impact trend, scope trend, confirmed findings, rejected hypotheses, leading hypotheses
with evidence, actions completed, actions in progress with owners, blockers, decisions
required now, and the next communication time. Keep unresolved statements explicitly
uncertain and flag any action that needs change or security approval.
```

ESCALATION HANDOFF

USE WHEN: another team, vendor, or senior responder must take over

```
Create a concise technical escalation from these incident records [records]. Include
service and environment, impact, timeline in one timezone, exact symptoms and errors,
evidence collected, actions attempted and results, current hypotheses, affected
dependencies, access or data-sensitivity limits, current mitigation, and the specific
question or action requested from the recipient. Remove speculation and duplicate events.
```

BEFORE YOU SUBMIT

- ✓ All times use one timezone
- ✓ Every action has an owner
- ✓ Facts are not mixed with hypotheses
- ✓ Impact and scope are separate
- ✓ Escalation triggers are measurable

TEMPLATE USE SEQUENCE

01	02	03	04	05
Replace fields	**Remove irrelevant text**	**Mark unknowns**	**Check data handling**	**Verify the output**

CHAPTER 04

Windows Systems

Prompt across identity, policy, services, permissions, and event evidence

DISCIPLINE CONTEXT

Windows faults often cross layers that look similar to the user. An access failure may begin with DNS or Kerberos, surface as a permissions error, and be made intermittent by policy or token state. The prompt should name the Windows edition, version, server role, domain relationship, service, and user journey.

Comparative evidence is especially powerful. Provide a working user, device, server, policy result, or event window beside the failing case. Ask the model to separate authentication, authorization, service health, policy application, and local operating-system state before it recommends a change.

PROMPT ENGINEERING LENS

WHY IT MATTERS

Windows advice becomes safer when commands are requested with purpose, expected output, privilege requirement, and whether they are read-only. Ask for PowerShell and GUI paths only when both are genuinely useful.

FIELD RULE

In Windows, identify the failing layer before interpreting the message shown to the user.

THE DISCIPLINE SEQUENCE

01	02	03	04	05
Identity	**Policy**	**Service**	**Permissions**	**Events**

INCLUDE IN THE PROMPT

- Edition, build, role, and domain context
- Affected user/device/server and a working comparison
- Relevant event IDs with source and timestamp
- Recent GPO, patch, permission, or service changes

AVOID THESE WEAK PATTERNS

- Restarting services as a first test
- Treating access denied as proof of NTFS failure
- Ignoring token refresh and replication timing
- Running broad repair commands without scope

MAKE THE REASONING VISIBLE

EVIDENCE	FALSIFY	CONTROL
What fact would most change the current ranking?	What result would disprove the leading cause?	Which next step is safe, reversible, and authorised?

OPERATIONAL CONTEXT

Windows investigations benefit from exact, time-aligned comparisons. Capture the client, server, selected domain controller, user, policy result, and event sources for one reproducible attempt. Ask the model to explain required privilege and read/write effect so a junior operator can collect evidence without accidentally changing policy, services, or permissions.

CHAPTER 04

Windows Systems

Prompt across identity, policy, services, permissions, and event evidence

WORKED FIELD EXAMPLE

SCENARIO

A Windows Server 2022 file server returns intermittent access-denied errors on one share. Most users are unaffected. Security events show occasional Kerberos warnings. Share and NTFS permissions have not yet been compared with a working user.

COPY-READY PROMPT

```
Act as a senior Windows engineer. Environment: Windows Server 2022 domain-joined file server; one SMB share is affected. Symptom: intermittent access denied for a small user group; most users work. Evidence: occasional Kerberos warnings in the Security log. Unknowns: whether the denial occurs at authentication, share authorization, or NTFS authorization. Constraint: read-only investigation during business hours. Build a comparison-led plan using one working and one failing user. Cover group membership, token state, share and NTFS effective access, inheritance, domain-controller selection, time/DNS, and correlated events. Give commands only with purpose, expected output, and interpretation.
```

WHY THIS PROMPT WORKS

- Does not assume the visible error identifies the layer
- Uses effective-access comparisons
- Correlates events rather than searching blindly
- Requests command meaning and privilege context

CONDENSED SENIOR ENGINEER RESPONSE

Confirm the failing user's current group token and compare it with a working user. Check share permissions and NTFS effective access separately, including inheritance and deny entries. Correlate a single failed attempt with Security, SMB, and Kerberos events on the client, server, and selected domain controller. Verify time and DNS because authentication warnings may be upstream of authorization. If the same token and path work intermittently, examine domain-controller selection, replication, cached credentials, and policy timing before changing permissions.

NEXT-TURN REFINEMENT 1

```
Turn this into an elevated, read-only PowerShell collection plan with no configuration changes.
```

NEXT-TURN REFINEMENT 2

```
Given these event records, build a timestamped authentication-to-authorization sequence and identify the missing evidence.
```

WHAT THIS EXAMPLE DOES NOT PROVE

A Kerberos, service, or access-denied event may be a symptom rather than the initiating fault. Read-only evidence also cannot prove an intermittent issue that was not reproduced in the captured window. Keep alternate layers open until the event chain and effective state agree.

TURN THE ANSWER INTO EVIDENCE

01	02	03	04	05
Confirm facts	**Run one safe check**	**Interpret result**	**Update the plan**	**Record the decision**

CHAPTER 04

3 / 3

Windows Systems

Prompt across identity, policy, services, permissions, and event evidence

PROMPT LIBRARY

Use these as starting structures. Replace every bracketed field, remove anything irrelevant, and keep unknowns explicit. The strongest template is the one that matches the real decision you need to make.

WINDOWS SERVICE FAILURE

USE WHEN: a Windows service will not start or stay running

```
Act as a senior Windows systems engineer. Host role/build: [details]. Service/name: [details]. Startup type/account: [details]. Symptom and exact error: [details]. Last known good: [time]. Recent patch/config/account changes: [details]. Relevant System/Application events: [sanitised entries]. Build a read-only diagnosis from service state and dependencies through logon rights, file/config access, ports, and resource state. For each command, state privilege, purpose, expected result, interpretation, and stop condition.
```

GROUP POLICY COMPARISON

USE WHEN: policy applies differently across users or devices

```
Act as a senior Windows and Group Policy engineer. Expected policy: [name/setting]. Affected user/device/OU: [details]. Working comparison: [details]. Domain/site/DC context: [details]. Resultant-set evidence: [gpresult or event summary]. Recent GPO, filtering, replication, or OU changes: [details]. Compare scope, link order, inheritance, security/WMI filtering, loopback, replication, and client processing. Ask for missing evidence before suggesting gpupdate, relinking, or policy edits.
```

FILE ACCESS INVESTIGATION

USE WHEN: SMB access is denied, slow, or inconsistent

```
Act as a senior Windows file-services engineer. Client/server versions: [details]. UNC path: [sanitised path]. Affected and working identities: [details]. Symptom: [denied/slow/intermittent]. Evidence: [SMB, Security, Kerberos, DNS, network]. Compare authentication, share permissions, NTFS effective access, inheritance, token/group state, DFS if used, endpoint security, and path latency. Order checks from read-only to higher risk and define what proves recovery.
```

BEFORE YOU SUBMIT

- ✓ Windows build and server role are named
- ✓ Events include source and timestamp
- ✓ A working comparison is present
- ✓ Authentication and authorization are separate
- ✓ Commands include expected interpretation

TEMPLATE USE SEQUENCE

01	02	03	04	05
Replace fields	**Remove irrelevant text**	**Mark unknowns**	**Check data handling**	**Verify the output**

CHAPTER 05

Linux Systems

Use distribution context, service state, logs, configuration, dependencies, and resources

DISCIPLINE CONTEXT

Linux guidance changes with distribution, release, init system, package source, security policy, and deployment model. State those details early. Include the exact unit, process, container, mount, path, exit status, and short log window rather than paraphrasing the error.

Ask for a layered path that starts with observable state: service status, journal or application logs, configuration validation, dependency availability, permissions and mandatory-access controls, and resource pressure. The model should explain why each command belongs in the sequence.

PROMPT ENGINEERING LENS

WHY IT MATTERS

Request commands that are non-destructive by default and compatible with the named distribution. Tell the model not to disable SELinux, AppArmor, a firewall, or certificate validation merely to make the symptom disappear.

FIELD RULE

Start with state and evidence; change one layer only after the failing layer is clear.

THE DISCIPLINE SEQUENCE

01	02	03	04	05
State	**Logs**	**Config**	**Dependencies**	**Resources**

INCLUDE IN THE PROMPT

- Distribution, release, kernel, and deployment type
- Exact service/unit, account, path, and error text
- Relevant journal time window and exit code
- Recent package, config, policy, mount, or certificate changes

AVOID THESE WEAK PATTERNS

- Generic commands for the wrong distribution
- Disabling security controls as a test
- Editing config before running its validator
- Ignoring boot order and mount availability

MAKE THE REASONING VISIBLE

EVIDENCE	FALSIFY	CONTROL
What fact would most change the current ranking?	What result would disprove the leading cause?	Which next step is safe, reversible, and authorised?

OPERATIONAL CONTEXT

Keep the distribution's supported tooling and security model visible throughout the prompt. Where possible, supply validator output before configuration content and a short journal window around one failed start. This lets the model reason from state and dependency order without defaulting to broad permission changes or disabling enforcement.

CHAPTER 05

Linux Systems

Use distribution context, service state, logs, configuration, dependencies, and resources

WORKED FIELD EXAMPLE

SCENARIO

On RHEL, an application service fails after reboot with permission denied. It worked before restart. The service depends on a mounted path and runs under a dedicated account. SELinux is enforcing.

COPY-READY PROMPT

```
Act as a senior RHEL support engineer. Environment: RHEL [version], systemd, SELinux enforcing. Service: [unit], running as [account]. Symptom: service fails only after reboot; journal shows permission denied. Dependency: application data should be mounted at [sanitised path]. Constraint: do not disable SELinux or change permissions until the failing layer is proven. Build a read-only diagnosis covering unit status and ordering, journal evidence, mount availability, ownership and mode, SELinux context/denials, configuration paths, and dependency health. For each check give the command, why it is safe, expected result, and next branch.
```

WHY THIS PROMPT WORKS

- Names the distribution and security mode
- Links the symptom to reboot and a dependency
- Prevents unsafe blanket permission changes
- Requests a branchable diagnostic sequence

CONDENSED SENIOR ENGINEER RESPONSE

Correlate unit status, journal entries, and mount state from the same boot. Confirm whether the service starts before the required mount is available and whether its unit declares that dependency. Compare file ownership, mode, and SELinux context with the last known good expectation, then review audit denials rather than disabling enforcement. Validate the application configuration without editing it. If the path is absent at service start, correct dependency ordering through the approved change process; if the path exists but access is denied, use the evidence to separate Unix permissions from SELinux policy.

NEXT-TURN REFINEMENT 1

```
Explain the difference between Unix permission denial and SELinux denial using only the evidence I provide.
```

NEXT-TURN REFINEMENT 2

```
Convert the confirmed cause into a minimal change plan with validation and rollback.
```

WHAT THIS EXAMPLE DOES NOT PROVE

Permission denied alone does not distinguish Unix mode, ownership, mount state, container mapping, SELinux, AppArmor, or application logic. The example narrows the possibilities but only boot-time unit, mount, audit, and file-context evidence can identify the responsible layer.

TURN THE ANSWER INTO EVIDENCE

01	02	03	04	05
Confirm facts	**Run one safe check**	**Interpret result**	**Update the plan**	**Record the decision**

CHAPTER 05

3 / 3

Linux Systems

Use distribution context, service state, logs, configuration, dependencies, and resources

PROMPT LIBRARY

Use these as starting structures. Replace every bracketed field, remove anything irrelevant, and keep unknowns explicit. The strongest template is the one that matches the real decision you need to make.

SYSTEMD SERVICE DIAGNOSIS

USE WHEN: a unit fails, restarts, or behaves differently after boot

```
Act as a senior Linux engineer for [distribution/version]. Unit: [name]. Service account: [account]. Symptom/exit code: [details]. Last known good: [time]. Recent changes: [package/config/unit/cert]. Logs from the same window: [sanitised entries]. Build a read-only sequence across unit state, dependencies/order, environment files, executable/config access, ports, security policy, and resources. Use distribution-appropriate commands and explain each result before proposing a change.
```

DISK AND RESOURCE PRESSURE

USE WHEN: a host is slow, full, or unstable

```
Act as a senior Linux operations engineer. Host/workload: [details]. Baseline: [normal state]. Current symptom: [details]. Evidence: [load, memory, filesystem, inode, I/O, process, cgroup/container metrics]. Identify whether pressure is CPU, memory, swap, storage latency, capacity, inode, file-handle, or workload contention. Give a safe measurement order, distinguish symptoms from causes, and do not recommend deleting files or killing processes without ownership, impact, and recovery checks.
```

PACKAGE OR CONFIG REGRESSION

USE WHEN: failure follows an update or configuration change

```
Act as a senior [distribution] engineer. Component/version before and after: [details]. Change method/repository: [details]. Symptom and first occurrence: [details]. Config validation output: [details]. Logs: [details]. Compare package files, dependencies, service environment, config syntax, permissions, and release notes supplied here. Build a confirmation plan, then a controlled forward-fix or rollback plan with backup, success, and abort criteria.
```

BEFORE YOU SUBMIT

- ✓ Distribution and release are explicit
- ✓ Security controls remain enabled
- ✓ Any change has validation and rollback
- ✓ Commands match the environment
- ✓ Logs and state use one time window

TEMPLATE USE SEQUENCE

01	02	03	04	05
Replace fields	**Remove irrelevant text**	**Mark unknowns**	**Check data handling**	**Verify the output**

CHAPTER 06

Networking and DNS

Separate physical path, routing, policy, name resolution, TLS, and application reachability

DISCIPLINE CONTEXT

Network prompts need exact endpoints. State source and destination, IP family, port, protocol, site or VLAN, expected route, and what succeeds. A successful ping proves only a narrow kind of reachability; it does not prove that DNS, firewall policy, TCP, TLS, proxy, or the application is healthy.

For DNS, include the queried name, record type, expected answer, resolver used, and whether the result differs by client, site, or time. Ask the model to preserve the layer boundary so that a name-resolution issue is not blended with a routing or application diagnosis.

PROMPT ENGINEERING LENS

WHY IT MATTERS

Use side-by-side path evidence from one working and one failing source. Ask for the shortest test sequence that confirms where the paths diverge, and require the model to state what each tool can and cannot prove.

FIELD RULE

Name the two endpoints and prove the first layer where their expected conversation stops.

THE DISCIPLINE SEQUENCE

01	02	03	04	05
Resolve	**Route**	**Policy**	**Transport**	**Application**

INCLUDE IN THE PROMPT

- Source, destination, port, protocol, and IP family
- Working path and failing path
- Resolver, DNS answer, TTL, route, and policy evidence
- Proxy, NAT, TLS, and application boundaries

AVOID THESE WEAK PATTERNS

- Using ping as complete proof
- Changing DNS and firewall at the same time
- Ignoring IPv6 or split DNS
- Pasting unsanitised packet captures

MAKE THE REASONING VISIBLE

EVIDENCE	FALSIFY	CONTROL
What fact would most change the current ranking?	What result would disprove the leading cause?	Which next step is safe, reversible, and authorised?

OPERATIONAL CONTEXT

Collect evidence from both sides of the path when tooling and ownership allow it. A client test shows what left or returned to the source; a firewall session shows policy and state; destination evidence shows receipt; an application log shows handling. Prompting should preserve those observation boundaries instead of treating every tool as an end-to-end proof.

CHAPTER 06

Networking and DNS

Separate physical path, routing, policy, name resolution, TLS, and application reachability

WORKED FIELD EXAMPLE

SCENARIO

Users on VLAN 30 cannot reach an internal HTTPS service. ICMP succeeds, but TCP 443 or the application connection fails. VLAN 20 works. A firewall policy changed earlier in the day and both VLANs resolve the same service address.

COPY-READY PROMPT

```
Act as a senior network engineer. Source A: VLAN 30 [subnet]; Source B: working VLAN 20 [subnet]. Destination: internal HTTPS service [sanitised FQDN/IP], TCP 443. Symptom: ICMP succeeds from both, HTTPS fails only from VLAN 30. DNS answer is identical. Recent change: firewall policy modified today. Give the shortest read-only path to confirmation across route, firewall session/policy, NAT if present, TCP handshake, TLS, and destination receipt. For each check, state where to run it, expected working/failing evidence, and what it proves. Do not treat the policy change as root cause until path evidence supports it.
```

WHY THIS PROMPT WORKS

- Defines both paths and the exact service
- Treats ping as limited evidence
- Uses the same DNS answer to lower one hypothesis
- Asks where the packet or session disappears

CONDENSED SENIOR ENGINEER RESPONSE

Compare route and next hop from both VLANs, then inspect firewall policy match and session creation for the same timestamped connection attempt. Confirm whether SYN traffic leaves VLAN 30, reaches the firewall, is allowed, and reaches the destination; then check whether a reply returns. If TCP completes, move to TLS and application evidence. Because VLAN 20 works and DNS is identical, a policy or path difference is stronger than a service-wide application fault, but it remains a hypothesis until the session evidence shows divergence.

NEXT-TURN REFINEMENT 1

```
Here are the route and firewall results. Identify the first confirmed point of divergence.
```

NEXT-TURN REFINEMENT 2

```
Explain what this DNS/TCP/TLS output proves and what evidence is still missing.
```

WHAT THIS EXAMPLE DOES NOT PROVE

Identical DNS answers and successful ICMP reduce some hypotheses but do not prove TCP, TLS, return routing, proxy policy, or application health. The example still needs timestamped session or packet evidence to establish the first point of divergence.

TURN THE ANSWER INTO EVIDENCE

01	02	03	04	05
Confirm facts	**Run one safe check**	**Interpret result**	**Update the plan**	**Record the decision**

CHAPTER 06

Networking and DNS

Separate physical path, routing, policy, name resolution, TLS, and application reachability

PROMPT LIBRARY

Use these as starting structures. Replace every bracketed field, remove anything irrelevant, and keep unknowns explicit. The strongest template is the one that matches the real decision you need to make.

REACHABILITY PATH

USE WHEN: one source cannot reach one service

```
Act as a senior network engineer. Source: [host/IP/VLAN/site]. Destination: [FQDN/IP].
Service: [protocol/port]. Expected path: [known hops/proxy/NAT]. Symptom:
[timeout/refused/reset/TLS/app error]. Working comparison: [source/path]. Evidence: [DNS,
route, TCP, firewall, capture summaries]. Build a layer-by-layer read-only plan. For each
test state location, command/tool, expected result, what it proves, and the next branch.
Minimise packet capture and sanitise addresses where required.
```

DNS INCONSISTENCY

USE WHEN: answers differ by client, resolver, site, or time

```
Act as a senior DNS engineer. Query name/type: [FQDN/record]. Expected answer: [value].
Failing client/resolver: [details]. Working client/resolver: [details]. Actual answers,
TTLs, and timestamps: [details]. DNS design:
[authoritative/forwarder/split-horizon/private zone]. Compare client cache, resolver
cache, forwarding, delegation, authoritative data, negative caching, and replication. Give
the minimum checks needed to locate the first inconsistent layer before proposing cache
clearing or record changes.
```

TLS OR APPLICATION EDGE

USE WHEN: TCP connects but HTTPS or another encrypted service fails

```
Act as a senior network and application-edge engineer. Client/source: [details].
FQDN/port: [details]. TCP result: [details]. TLS error/certificate chain/SNI: [sanitised
output]. Proxy/load balancer path: [details]. Working comparison: [details]. Separate TCP,
TLS negotiation, certificate trust/name/time, proxy policy, backend health, and
application response. Rank checks, explain what each layer proves, and avoid disabling
certificate validation as a fix.
```

BEFORE YOU SUBMIT

- ✓ Endpoints and port are exact
- ✓ DNS is tested separately
- ✓ Captures and addresses are minimised
- ✓ A working path is included
- ✓ Each tool has a proof boundary

TEMPLATE USE SEQUENCE

01	02	03	04	05
Replace fields	**Remove irrelevant text**	**Mark unknowns**	**Check data handling**	**Verify the output**

CHAPTER 07

1 / 3

Identity and Access

Trace the user journey from sign-in through policy, token, claims, and entitlement

DISCIPLINE CONTEXT

Identity incidents become manageable when the prompt identifies the failure stage. Primary authentication, multifactor authentication, conditional access, token issuance, role or group claims, and application authorization are different checkpoints even when the user reports only cannot log in.

Provide a working identity path for comparison and the exact timestamp, application, tenant, client, error code, and policy result. Ask the model to distinguish identity proof from access entitlement and to account for replication, token lifetime, cached sessions, and application-side mapping.

PROMPT ENGINEERING LENS

WHY IT MATTERS

Never paste passwords, recovery codes, tokens, session cookies, private keys, or full personal records. Use IDs only when needed and sanitise sign-in evidence. Ask for policy interpretation, not a bypass.

FIELD RULE

Find the last successful identity checkpoint and investigate the next one.

THE DISCIPLINE SEQUENCE

01	02	03	04	05
Authenticate	**Evaluate**	**Issue token**	**Map claims**	**Authorise**

INCLUDE IN THE PROMPT

- Tenant, application, client type, and timestamp
- Exact error and sign-in or audit result
- Affected identity and a working comparison
- Group, role, licence, policy, and token-change timing

AVOID THESE WEAK PATTERNS

- Calling every access issue an MFA issue
- Using global-admin access as a test
- Ignoring stale tokens or app-side roles
- Sharing credentials or bearer tokens

MAKE THE REASONING VISIBLE

EVIDENCE	FALSIFY	CONTROL
What fact would most change the current ranking?	What result would disprove the leading cause?	Which next step is safe, reversible, and authorised?

OPERATIONAL CONTEXT

Build a checkpoint table for the working and failing identities. Record authentication, policy evaluation, token issue time, expected claims, assignment, application mapping, and final authorization. This prevents broad privilege changes and makes propagation or cached-session explanations visible without requiring the operator to share a live token.

CHAPTER 07

Identity and Access

Trace the user journey from sign-in through policy, token, claims, and entitlement

WORKED FIELD EXAMPLE

SCENARIO

A user signs in successfully to an Entra-integrated application but receives not authorized inside the app. A colleague in the same team works. Access is group-based and a membership change occurred recently.

COPY-READY PROMPT

```
Act as a senior identity engineer. Platform: Microsoft Entra ID. Application: [name/type]. Affected user signs in successfully but receives not authorized in the application. Working comparison: a colleague with expected access. Access model: group-based assignment; membership changed at [time]. Evidence: sign-in success and [sanitised correlation/error details]. Build a comparison plan across group membership, assignment, licence, token issuance time, claims, application role mapping, consent if relevant, and app-side entitlement. Separate what is confirmed at authentication from what must be proven at authorization. Do not request tokens or recommend broad privilege.
```

WHY THIS PROMPT WORKS

- Starts after the confirmed sign-in stage
- Uses a working peer as the baseline
- Includes timing for membership and token state
- Checks both identity-provider and application mapping

CONDENSED SENIOR ENGINEER RESPONSE

Treat primary authentication as confirmed. Compare the two users across direct and transitive group membership, enterprise-app assignment, licence, and application-role mapping. Establish whether the affected user's current token was issued before the membership change and whether the expected group or role claim is present using approved diagnostic views, not by sharing the token. If the claim is present but access still fails, move to the application's entitlement mapping and local user record. If it is absent, investigate assignment, group type, propagation, token refresh, and claim configuration.

NEXT-TURN REFINEMENT 1

```
Build a fact table showing the last successful checkpoint for the working and failing users.
```

NEXT-TURN REFINEMENT 2

```
Review this conditional-access result without suggesting a bypass or exclusion.
```

WHAT THIS EXAMPLE DOES NOT PROVE

Two colleagues in the same team may have different direct assignments, group paths, licences, token ages, or application records. A missing claim can reflect configuration or timing; a present claim does not prove the application has mapped it correctly. Both sides of the trust boundary require evidence.

TURN THE ANSWER INTO EVIDENCE

01	02	03	04	05
Confirm facts	**Run one safe check**	**Interpret result**	**Update the plan**	**Record the decision**

CHAPTER 07

Identity and Access

Trace the user journey from sign-in through policy, token, claims, and entitlement

PROMPT LIBRARY

Use these as starting structures. Replace every bracketed field, remove anything irrelevant, and keep unknowns explicit. The strongest template is the one that matches the real decision you need to make.

SIGN-IN FAILURE

USE WHEN: authentication or policy evaluation fails

```
Act as a senior identity engineer. Provider/tenant: [details]. App/resource/client: [details]. User/device/location context: [minimised details]. Failure time/timezone and error code: [details]. Sign-in and policy result summary: [sanitised]. Working comparison: [details]. Trace credential validation, MFA, device state, risk, conditional-access evaluation, token issuance, and provider health. Rank evidence gaps and safe checks. Do not ask for passwords, codes, tokens, or policy bypass.
```

AUTHORIZATION COMPARISON

USE WHEN: sign-in succeeds but access is denied

```
Act as a senior access engineer. Authentication status: confirmed successful at [time]. Resource/action denied: [details]. Access model: [group/role/claim/app entitlement]. Affected identity: [sanitised]. Working identity: [sanitised]. Changes and timing: [details]. Compare assignment, group/role membership, token issuance and claims, licence, app mapping, and local entitlement. Show the last successful checkpoint and the first unproven checkpoint.
```

CONDITIONAL ACCESS REVIEW

USE WHEN: a policy appears to block or challenge a user

```
Act as a senior Entra security engineer. Sign-in context: [app, client, device, location, time]. Policy result: [sanitised summary]. Expected behaviour: [details]. Working comparison: [details]. Review policy targeting, exclusions, authentication strength, device/compliance, location, risk, grant/session controls, and report-only results. Explain the evaluation order and evidence. Recommend only policy-compliant corrections with approval, validation, and rollback.
```

BEFORE YOU SUBMIT

- ✓ The failure stage is explicit
- ✓ Change and token timing are included
- ✓ No broad privilege is used as a shortcut
- ✓ A working identity path exists
- ✓ No credentials or tokens are shared

TEMPLATE USE SEQUENCE

01	02	03	04	05
Replace fields	**Remove irrelevant text**	**Mark unknowns**	**Check data handling**	**Verify the output**

CHAPTER 08

Cloud and SaaS

Use provider evidence, service boundaries, deployment history, and shared responsibility

DISCIPLINE CONTEXT

Cloud and SaaS prompts need provider-specific scope: tenant or subscription, service, region, resource, deployment model, and management boundary. Separate the control plane, where configuration and deployments occur, from the data plane, where users and workloads make requests.

Start with provider-native evidence such as service health, activity or audit logs, deployment history, resource metrics, and instance comparisons. Then examine dependencies the service still relies on: identity, DNS, network integration, certificates, secrets, quotas, storage, and upstream or downstream APIs.

PROMPT ENGINEERING LENS

WHY IT MATTERS

Tell the model which parts you manage and which belong to the provider. Ask it to avoid host-level advice for a managed service unless that layer is genuinely exposed. Keep tenant IDs, keys, connection strings, and customer data out of prompts.

FIELD RULE

Start at the managed-service boundary and move outward only when provider evidence points there.

THE DISCIPLINE SEQUENCE

01	02	03	04	05
Scope	**Provider health**	**Control plane**	**Data plane**	**Dependencies**

INCLUDE IN THE PROMPT

- Provider, service, region, tenant/subscription, and SKU
- Healthy and failing resources or instances
- Activity/deployment timeline and provider errors
- Identity, network, quota, certificate, and data dependencies

AVOID THESE WEAK PATTERNS

- Assuming every error is a provider outage
- Using host troubleshooting on opaque SaaS
- Ignoring quotas and service limits
- Sharing keys, connection strings, or customer data

MAKE THE REASONING VISIBLE

EVIDENCE	FALSIFY	CONTROL
What fact would most change the current ranking?	What result would disprove the leading cause?	Which next step is safe, reversible, and authorised?

OPERATIONAL CONTEXT

Use provider-native identifiers and timestamps in the approved support record, while keeping secrets and customer data out of the prompt. Compare resources at the smallest meaningful boundary: instance, slot, region, tenant, subscription, policy, or deployment. Then ask whether the fault follows the workload, configuration, dependency, or provider placement.

CHAPTER 08

Cloud and SaaS

Use provider evidence, service boundaries, deployment history, and shared responsibility

WORKED FIELD EXAMPLE

SCENARIO

After a deployment, one Azure App Service instance returns server errors while another stays healthy. Startup logs on the failing instance show a dependency timeout. No regional service-health event is visible.

COPY-READY PROMPT

```
Act as a senior Azure platform engineer. Service: Azure App Service
[OS/runtime/SKU/region]. Change: deployment at [time]. Symptom: one instance returns HTTP
5xx; another instance in the same plan remains healthy. Evidence: failing-instance startup
log shows dependency timeout; regional service health is clear. Constraint: use read-only
provider-native evidence before restart, scale, or redeployment. Build a targeted
comparison across deployment state, app settings and slot settings, managed identity,
network integration/DNS, dependency reachability, certificates/secrets references,
instance metrics, and platform events. State what would justify recycle, rollback, or
provider escalation.
```

WHY THIS PROMPT WORKS

- Uses the healthy instance as a control
- Separates regional health from resource health
- Begins with provider-native evidence
- Defines thresholds for higher-risk actions

CONDENSED SENIOR ENGINEER RESPONSE

Compare deployment and configuration state at instance and slot boundaries, then align startup timestamps with dependency, identity, DNS, and network evidence. Confirm whether the timeout is unique to the failing instance or follows a setting, route, or dependency. Review platform events and instance metrics before recycling, because a restart may erase useful state without fixing the cause. Roll back only if the deployment is proven to introduce the failure and the previous version remains compatible; escalate to the provider if the fault follows platform placement and tenant-controlled dependencies are healthy.

NEXT-TURN REFINEMENT 1

```
Convert these Azure activity and
application logs into a
control-plane/data-plane timeline.
```

NEXT-TURN REFINEMENT 2

```
List the exact evidence needed before
choosing restart, rollback, scale, or
provider escalation.
```

WHAT THIS EXAMPLE DOES NOT PROVE

A clear regional health dashboard does not prove every resource placement is healthy. A working instance also may not share the same connection, route, cache, secret version, or dependency timing. The comparison guides the next evidence request; it does not by itself prove the cause.

TURN THE ANSWER INTO EVIDENCE

01	02	03	04	05
Confirm facts	**Run one safe check**	**Interpret result**	**Update the plan**	**Record the decision**

CHAPTER 08

Cloud and SaaS

Use provider evidence, service boundaries, deployment history, and shared responsibility

PROMPT LIBRARY

Use these as starting structures. Replace every bracketed field, remove anything irrelevant, and keep unknowns explicit. The strongest template is the one that matches the real decision you need to make.

POST-DEPLOYMENT FAILURE

USE WHEN: a cloud service degrades after a release

```
Act as a senior [provider] platform engineer. Tenant/subscription/region: [sanitised].
Service/SKU/runtime: [details]. Deployment time/method/version: [details]. Symptom and
scope: [details]. Healthy comparison: [resource/instance/region]. Provider
health/activity/deployment evidence: [summary]. Build a read-only comparison across
configuration, identity, network/DNS, secrets/certificates references, quotas,
dependencies, metrics, and platform events. Define proof for forward-fix, rollback,
restart, scale, and provider escalation.
```

SAAS SERVICE INCIDENT

USE WHEN: users report failure in a provider-managed application

```
Act as a senior SaaS support engineer. Product/tenant/feature: [details]. Affected and
unaffected users/locations/clients: [details]. Start time/timezone: [details]. Provider
health/advisory status: [details]. Errors and audit evidence: [sanitised]. Recent
tenant-side changes: [details]. Separate provider service, tenant configuration, identity,
client, network edge, licence, and data-scope hypotheses. Give minimal confirmation steps
and a vendor-ready escalation pack.
```

CLOUD CONFIGURATION DRIFT

USE WHEN: two environments or resources behave differently

```
Act as a senior cloud configuration reviewer. Expected baseline: [policy/IaC/template].
Working resource/environment: [details]. Failing resource/environment: [details]. Known
differences: [details]. Activity/audit history: [summary]. Compare only behaviourally
relevant configuration across identity, policy, network, runtime, dependencies,
SKU/limits, and deployment artefacts. Separate intentional exceptions from unexplained
drift and propose a controlled reconciliation with validation and rollback.
```

BEFORE YOU SUBMIT

- ✓ Provider scope and region are named
- ✓ A healthy resource is compared
- ✓ Secrets and customer data are excluded
- ✓ Control and data planes are separate
- ✓ Provider-native evidence leads

TEMPLATE USE SEQUENCE

01	02	03	04	05
Replace fields	**Remove irrelevant text**	**Mark unknowns**	**Check data handling**	**Verify the output**

CHAPTER 09

Logs, Metrics, and Traces

Ask the model to reconstruct an event sequence, not merely repeat raw telemetry

DISCIPLINE CONTEXT

Observability prompts should begin with a question. State the incident window, timezone, systems involved, normal baseline, and the decision you need the evidence to support. Without that frame, the model may summarise noisy entries instead of finding the first meaningful deviation.

Use multiple sources to build sequence: logs describe events, metrics show behaviour over time, and traces connect work across services. Ask the model to identify timestamp alignment, repeated symptoms, precursor events, missing spans, and evidence that would disprove the leading story.

PROMPT ENGINEERING LENS

WHY IT MATTERS

Sanitise personal data, tokens, URLs, payloads, and internal identifiers before sharing. Sample only the relevant window and preserve original records in the approved system. The model's summary is not the authoritative evidence store.

FIELD RULE

Find the first abnormal event that explains the later pattern, then ask what would falsify it.

THE DISCIPLINE SEQUENCE

01	02	03	04	05
Question	**Time-align**	**Cluster**	**Sequence**	**Confirm**

INCLUDE IN THE PROMPT

- Time window and one timezone
- Source, host/service, severity, and correlation ID
- Baseline and known-good comparison
- Sampling gaps, clock skew, retention, and redaction

AVOID THESE WEAK PATTERNS

- Pasting huge unbounded log sets
- Treating the loudest error as the first cause
- Ignoring clock or timezone differences
- Claiming causation from proximity alone

MAKE THE REASONING VISIBLE

EVIDENCE	FALSIFY	CONTROL
What fact would most change the current ranking?	What result would disprove the leading cause?	Which next step is safe, reversible, and authorised?

OPERATIONAL CONTEXT

Prepare evidence deliberately: one decision question, one timezone, a bounded incident window, source labels, and a healthy baseline. Preserve raw telemetry in its original platform and give the model only the sanitised slice needed to build the sequence. This makes the analysis repeatable without turning the AI thread into an uncontrolled evidence store.

CHAPTER 09

Logs, Metrics, and Traces

Ask the model to reconstruct an event sequence, not merely repeat raw telemetry

WORKED FIELD EXAMPLE

SCENARIO

An API incident has reverse-proxy logs, application logs, dependency traces, and latency/error metrics from the same 20-minute window. Proxy 502 errors rise after dependency latency increases, but the first abnormal event is not yet known.

COPY-READY PROMPT

```
Act as a senior observability engineer. Question: what event sequence best explains the API failures, and what evidence would disprove it? Window: [start-end, UTC]. Sources: reverse-proxy logs, application logs, distributed traces, and service/dependency metrics. Baseline: [normal latency/error rate]. I will provide sanitised extracts with source and timestamp. Align time, note possible clock skew, group repeated symptoms, identify the earliest meaningful deviation, connect events by request/correlation ID where available, and distinguish correlation from causation. Return a timeline, leading and alternative hypotheses, confidence labels, and the next missing signal.
```

WHY THIS PROMPT WORKS

- Defines the analytical question
- Combines sources with a common time window
- Looks for the first deviation, not the loudest event
- Requires alternative explanations and falsification

CONDENSED SENIOR ENGINEER RESPONSE

Normalise timestamps and mark any source with uncertain clock alignment. Plot dependency latency, application duration, and proxy errors on the same sequence. Use traces to determine whether slow dependency spans precede application timeouts and 502 responses for the same requests. Repeated proxy errors are likely symptoms if they begin after upstream latency. Confirm the story with request-level linkage and a healthy-period comparison; disprove it if failed requests show normal dependency timing or if the first abnormal event occurs in the proxy or application before the dependency change.

NEXT-TURN REFINEMENT 1

Separate precursor events, primary failure, propagated symptoms, and recovery signals.

NEXT-TURN REFINEMENT 2

Show which conclusions are supported by request-level evidence and which rely only on time correlation.

WHAT THIS EXAMPLE DOES NOT PROVE

Sampling, missing spans, retry storms, aggregation, and clock skew can reorder or exaggerate the apparent story. Even a persuasive timeline is not causal proof unless request-level or controlled evidence links the precursor to the failure and alternative sequences are genuinely contradicted.

TURN THE ANSWER INTO EVIDENCE

01	02	03	04	05
Confirm facts	**Run one safe check**	**Interpret result**	**Update the plan**	**Record the decision**

CHAPTER 09

3 / 3

Logs, Metrics, and Traces

Ask the model to reconstruct an event sequence, not merely repeat raw telemetry

PROMPT LIBRARY

Use these as starting structures. Replace every bracketed field, remove anything irrelevant, and keep unknowns explicit. The strongest template is the one that matches the real decision you need to make.

MULTI-SOURCE TIMELINE

USE WHEN: you need one incident story from several telemetry sources

```
Act as a senior observability engineer. Decision question: [question]. Incident
window/timezone: [details]. Baseline: [normal]. Sources and clock notes: [list]. Sanitised
records: [extracts]. Normalise time, group duplicates, connect correlation IDs, identify
the earliest deviation, and produce a timeline of precursor, failure, propagation,
mitigation, and recovery. Label facts, inferences, confidence, contradictions, and the
next evidence that could disprove the leading sequence.
```

LOG-PATTERN TRIAGE

USE WHEN: a manageable log sample contains many repeated events

```
Review these sanitised logs as a senior operations engineer. System and question:
[details]. Window/timezone: [details]. Baseline or healthy sample: [details]. Group
messages by signature without hiding meaningful parameter differences. Count frequency and
first/last occurrence, identify changes around the first fault, separate root signals from
retries or cascades, and list the three highest-value follow-up queries. Do not invent
missing fields or decode secrets.
```

METRIC AND TRACE CORRELATION

USE WHEN: latency or errors cross several services

```
Act as a senior performance and tracing engineer. User symptom/SLO: [details]. Services and
dependency graph: [summary]. Metric changes: [time series summary]. Trace sample:
[sanitised spans]. Compare baseline and incident percentiles, throughput, errors,
saturation, and span contribution. Identify where additional time or failure first appears,
state correlation limits, and request the smallest trace or metric needed to confirm the
bottleneck.
```

BEFORE YOU SUBMIT

- ✓ The analytical question is stated
- ✓ A baseline is available
- ✓ Causation and correlation are separate
- ✓ Time and source labels are consistent
- ✓ Sensitive fields are removed

TEMPLATE USE SEQUENCE

01	02	03	04	05
Replace fields	**Remove irrelevant text**	**Mark unknowns**	**Check data handling**	**Verify the output**

CHAPTER 10

Performance, Storage, and Backup

Use baselines, workload context, bottleneck evidence, capacity trends, and recovery proof

DISCIPLINE CONTEXT

Performance has meaning only against a baseline and workload. Provide normal and current response time, throughput, concurrency, percentile, and the layer where delay is observed. Low average CPU does not rule out queueing, single-thread limits, storage latency, locks, worker exhaustion, or a slow dependency.

For storage and backup, distinguish capacity, throughput, latency, errors, protection state, and recoverability. A successful job is not proof that a restore will meet the required recovery point and recovery time. Ask for evidence from restore tests and application-consistent validation.

PROMPT ENGINEERING LENS

WHY IT MATTERS

Prompt for measurement before tuning. Ask the model to identify the missing metric that would locate time or pressure, and to avoid optimisation advice that changes several layers without a controlled comparison.

FIELD RULE

Measure where time or risk accumulates; do not infer it from the noisiest host metric.

THE DISCIPLINE SEQUENCE

01	02	03	04	05
Baseline	**Workload**	**Measure**	**Locate**	**Recover**

INCLUDE IN THE PROMPT

- Normal versus current user-facing measure
- Workload, concurrency, size, and time pattern
- Compute, memory, network, storage, queue, and dependency evidence
- Backup scope, RPO/RTO, job stage, and last tested restore

AVOID THESE WEAK PATTERNS

- Using averages alone
- Equating free capacity with healthy latency
- Tuning before locating the delay
- Calling a green backup job recoverable

MAKE THE REASONING VISIBLE

EVIDENCE	FALSIFY	CONTROL
What fact would most change the current ranking?	What result would disprove the leading cause?	Which next step is safe, reversible, and authorised?

OPERATIONAL CONTEXT

Performance prompts should follow one unit of work from user action to completion, then compare it with a representative baseline. Storage and backup prompts should include trend and recovery objectives, not only current status. Ask the model to name the metric that accounts for missing time, capacity risk, or recovery uncertainty.

CHAPTER 10

2 / 3

Performance, Storage, and Backup

Use baselines, workload context, bottleneck evidence, capacity trends, and recovery proof

WORKED FIELD EXAMPLE

SCENARIO

A reporting application normally loads in 4 seconds and now takes 28. Application CPU is moderate, database CPU is low, and network round-trip time is normal. No request breakdown or storage latency has yet been captured.

COPY-READY PROMPT

```
Act as a senior performance engineer. Service: reporting application. Baseline: typical page load 4 seconds at [normal concurrency]. Current: 28 seconds at [current concurrency]. Evidence: app CPU moderate, database CPU low, network RTT normal. Unknown: where the extra 24 seconds is spent. Build a measurement plan across request timing, queue/worker saturation, database query duration and waits, cache behaviour, storage latency, dependency calls, and client rendering if relevant. Prioritise measurements that attribute elapsed time. For each metric state baseline comparison, interpretation, and the next branch. Do not recommend tuning until the bottleneck is demonstrated.
```

WHY THIS PROMPT WORKS

- Quantifies the user-facing regression
- Keeps low CPU from ending the investigation
- Requests elapsed-time attribution
- Delays tuning until evidence locates the bottleneck

CONDENSED SENIOR ENGINEER RESPONSE

Instrument one slow request from entry to response and break its 28 seconds into queue, application, database, dependency, storage, and client components. Check worker and connection-pool wait time even if CPU is low. Compare query duration and waits with the four-second baseline, then align storage and cache signals with the same request. The most useful next measurement is the one that accounts for the missing time; host utilisation is supporting context, not the answer.

NEXT-TURN REFINEMENT 1

```
Given this timing breakdown, rank the bottlenecks by contribution and evidence strength.
```

NEXT-TURN REFINEMENT 2

```
Design a controlled test that changes one performance variable and defines a measurable success threshold.
```

WHAT THIS EXAMPLE DOES NOT PROVE

One slow request may not represent the workload, and average utilisation can hide short saturation or a constrained queue. A completed backup or repository check cannot prove application recovery. Restore validation must be performed through the approved recovery process against stated RPO and RTO objectives.

TURN THE ANSWER INTO EVIDENCE

01	02	03	04	05
Confirm facts	**Run one safe check**	**Interpret result**	**Update the plan**	**Record the decision**

CHAPTER 10

3 / 3

Performance, Storage, and Backup

Use baselines, workload context, bottleneck evidence, capacity trends, and recovery proof

PROMPT LIBRARY

Use these as starting structures. Replace every bracketed field, remove anything irrelevant, and keep unknowns explicit. The strongest template is the one that matches the real decision you need to make.

APPLICATION SLOWNESS

USE WHEN: response time regresses or varies

```
Act as a senior performance engineer. Service/path: [details]. Baseline and current
p50/p95/p99 or measured times: [details]. Workload/concurrency/data size: [details].
Affected scope/time pattern: [details]. Evidence: [CPU, memory, queues, pools, DB waits,
storage, network, dependencies, traces]. Build an attribution plan that locates elapsed
time. State the baseline, expected signal, interpretation, and next test. Do not tune
before a bottleneck is demonstrated.
```

STORAGE CAPACITY AND I/O

USE WHEN: storage is slow, filling, or reporting errors

```
Act as a senior storage engineer. Platform/volume/workload: [details]. Capacity and
growth: [used/free/trend]. Performance baseline/current: [latency, IOPS, throughput,
queue]. Errors/path/redundancy: [summary]. Recent changes: [details]. Separate capacity,
inode/object limits, latency, throughput, queueing, path health, controller/service
limits, and workload change. Propose read-only checks, forecast breach dates with stated
assumptions, and define escalation before any cleanup or expansion.
```

BACKUP AND RECOVERY REVIEW

USE WHEN: a backup fails or recoverability must be proven

```
Act as a senior backup and recovery engineer. Protected workload: [details].
Policy/schedule/retention: [details]. Required RPO/RTO: [details]. Failure
stage/error/time: [details]. Last successful job and last tested restore: [details].
Dependencies: [repository, network, credentials, snapshots, application writers]. Diagnose
the job by stage, preserve evidence, and define a recovery-validation test. Distinguish
backup completion from application-consistent recoverability and include
escalation/rollback for any configuration change.
```

BEFORE YOU SUBMIT

- ✓ A normal baseline is quantified
- ✓ Time or pressure is attributed by layer
- ✓ Any tuning has one measurable target
- ✓ Workload and concurrency are included
- ✓ Backup success and restore proof are separate

TEMPLATE USE SEQUENCE

01	02	03	04	05
Replace fields	**Remove irrelevant text**	**Mark unknowns**	**Check data handling**	**Verify the output**

CHAPTER 11

1 / 3

Automation and Scripting

Prompt for safe behaviour, testability, observability, and controlled execution

DISCIPLINE CONTEXT

A script prompt is a miniature specification. Define language and version, operating environment, inputs, outputs, authentication model, permissions, scale, failure behaviour, logging, and deployment method. State whether the task is inventory, recommendation, or remediation.

Ask for read-only or dry-run behaviour first, input validation, idempotency where appropriate, bounded concurrency, and an explicit approval gate before changes. A plain-language explanation and test plan make generated code reviewable by someone other than its author.

PROMPT ENGINEERING LENS

WHY IT MATTERS

Treat generated code as untrusted until reviewed and tested. Request no hard-coded secrets, least privilege, dependency checks, meaningful exit codes, and sample outputs. Ask the model to identify what it cannot safely infer about the target environment.

FIELD RULE

If the script can change many systems, the prompt must make a safe small test easier than a broad run.

THE DISCIPLINE SEQUENCE

01	02	03	04	05
Input	**Validate**	**Simulate**	**Execute**	**Report**

INCLUDE IN THE PROMPT

- Language/runtime and supported platforms
- Inputs, outputs, limits, and expected scale
- Auth, permissions, secret source, and audit requirements
- Dry run, test fixtures, error handling, rollback, and exit codes

AVOID THESE WEAK PATTERNS

- One-line scripts for broad change
- Hard-coded credentials or tenant values
- Silent catch-all error handling
- Unbounded parallelism or destructive defaults

MAKE THE REASONING VISIBLE

EVIDENCE	FALSIFY	CONTROL
What fact would most change the current ranking?	What result would disprove the leading cause?	Which next step is safe, reversible, and authorised?

OPERATIONAL CONTEXT

Generate design, tests, and code as separate review stages. Keep target selection explicit, make dry-run output easy to read, and ensure every target produces a result even when it fails. Store credentials in the approved mechanism and test permission, timeout, partial failure, and rerun behaviour before any broad or scheduled execution.

CHAPTER 11

Automation and Scripting

Prompt for safe behaviour, testability, observability, and controlled execution

WORKED FIELD EXAMPLE

SCENARIO

A PowerShell tool must check whether a named service is running across a server list and write a report. It should not start or stop anything, must handle offline hosts, and will first be tested against three known servers.

COPY-READY PROMPT

```
Act as a senior PowerShell automation engineer. Build a PowerShell 7-compatible, read-only tool that accepts a validated server list and service name, queries service state, and writes timestamped CSV and readable console output. It must make no changes. Requirements: least-privilege guidance, no embedded credentials, connectivity precheck, per-host timeout, structured error categories, bounded concurrency suitable for [scale], deterministic exit codes, and a summary of success/offline/denied/not-found. First provide design and pseudocode, then code, then Pester test cases and a three-host safe test plan. Explain each function and list assumptions.
```

WHY THIS PROMPT WORKS

- Separates design, code, and tests
- Defines read-only behaviour and scale
- Makes failures visible and classifiable
- Builds a small test path into the deliverable

CONDENSED SENIOR ENGINEER RESPONSE

Structure the tool into parameter validation, target normalisation, connectivity and permission-aware queries, result objects, reporting, and exit handling. Use a bounded worker limit rather than launching against every server at once. Return one record per target even when a host is offline or access is denied. Test parsing, service-found, service-missing, timeout, and access-denied paths with mocks before the three-host live check. Document exactly which permissions and remoting settings are required; do not attempt to enable them automatically.

NEXT-TURN REFINEMENT 1

```
Review the code for unintended writes, secret exposure, race conditions, and inconsistent output objects.
```

NEXT-TURN REFINEMENT 2

```
Add an approval boundary for optional remediation while keeping read-only mode the default.
```

WHAT THIS EXAMPLE DOES NOT PROVE

Mocks and small tests do not prove production authentication, network behaviour, scale, API limits, or every data shape. Generated code may also contain a valid-looking command with the wrong version semantics. Review dependencies and run the smallest authorised live test before expanding scope.

TURN THE ANSWER INTO EVIDENCE

01	02	03	04	05
Confirm facts	**Run one safe check**	**Interpret result**	**Update the plan**	**Record the decision**

CHAPTER 11

3 / 3

Automation and Scripting

Prompt for safe behaviour, testability, observability, and controlled execution

PROMPT LIBRARY

Use these as starting structures. Replace every bracketed field, remove anything irrelevant, and keep unknowns explicit. The strongest template is the one that matches the real decision you need to make.

READ-ONLY INVENTORY TOOL

USE WHEN: you need safe discovery across systems

```
Act as a senior [PowerShell/Python] automation engineer. Goal: collect [data] from
[targets]. Runtime/platform: [details]. Inputs and scale: [details]. Output/schema:
[details]. Auth and least privilege: [details]. Build read-only mode only. Include
validation, dependency checks, timeouts, bounded concurrency, structured errors, logging,
deterministic exit codes, sample output, unit tests/mocks, and a small live-test plan. No
hard-coded secrets or automatic environment changes. Explain assumptions and every
external dependency.
```

APPROVED REMEDIATION

USE WHEN: a tool may make a controlled change

```
Design a two-phase automation for [change]. Phase 1 must discover, validate prerequisites,
calculate impact, and produce a reviewable plan with no writes. Phase 2 may run only with
explicit approval [mechanism], scoped targets, and a change ID. Require idempotency,
before-state capture, per-target audit, stop-on-threshold behaviour, rollback or
compensating action, post-change validation, and safe resume. Provide threat/failure
analysis and tests before implementation.
```

GENERATED-CODE REVIEW

USE WHEN: AI or a colleague has produced a script

```
Review this [language/version] code as a senior automation and security engineer: [code].
Intended environment and privilege: [details]. Identify syntax/runtime issues, hidden
writes, destructive paths, injection risk, secret handling, excessive privilege, unbounded
concurrency, race conditions, weak error handling, poor observability, dependency
assumptions, and missing tests. Rank findings by severity, show the smallest safe
correction, and produce a revised test plan before revised code.
```

BEFORE YOU SUBMIT

- ✓ Read-only is the default where possible
- ✓ Inputs and scale are bounded
- ✓ Secrets are external and least privilege is stated
- ✓ Failures remain visible in output
- ✓ Tests precede broad execution

TEMPLATE USE SEQUENCE

01	02	03	04	05
Replace fields	**Remove irrelevant text**	**Mark unknowns**	**Check data handling**	**Verify the output**

CHAPTER 12

Change Planning

Turn technical intent into a controlled, reviewable, and reversible runbook

DISCIPLINE CONTEXT

Change prompts work best when they describe the desired outcome and the environment rather than jumping straight to commands. Include scope, dependencies, business impact, maintenance window, approvals, backups, owners, and monitoring. Ask for preparation, implementation, validation, rollback, communication, and closure as distinct phases.

Success and abort criteria must be measurable before the window begins. Ask the model to challenge the plan for hidden prerequisites, single points of failure, sequence mistakes, unsupported assumptions, and rollback steps that are slower or riskier than the change itself.

PROMPT ENGINEERING LENS

WHY IT MATTERS

AI can draft a runbook but cannot grant authority or confirm that a backup is valid. Request placeholders for local approvals and owners. Keep environment-specific values visible so reviewers can verify them.

FIELD RULE

A change is not ready until success, failure, and the decision to stop are all observable.

THE DISCIPLINE SEQUENCE

01	02	03	04	05
Prepare	**Precheck**	**Change**	**Validate**	**Rollback/close**

INCLUDE IN THE PROMPT

- Outcome, scope, dependencies, owners, and window
- Risk, impact, approvals, backups, and communications
- Exact prechecks, sequence, success, and abort criteria
- Rollback feasibility, duration, data implications, and closure evidence

AVOID THESE WEAK PATTERNS

- A rollback that is simply undo the change
- Validation limited to service started
- Missing decision owner
- Continuing after an abort threshold

MAKE THE REASONING VISIBLE

EVIDENCE	FALSIFY	CONTROL
What fact would most change the current ranking?	What result would disprove the leading cause?	Which next step is safe, reversible, and authorised?

OPERATIONAL CONTEXT

A runbook should be usable by an operator who did not write it. Use exact targets, owners, evidence fields, hold points, and decision wording. Tabletop the plan with the teams that own dependencies, then update the prompt with their constraints. The final record should preserve approvals and actual results separately from the AI draft.

CHAPTER 12

2 / 3

Change Planning

Turn technical intent into a controlled, reviewable, and reversible runbook

WORKED FIELD EXAMPLE

SCENARIO

Two domain controllers require patching in one maintenance window. Identity services must remain available. Replication, backup, DNS, time, and dependent services must be healthy before the first controller is changed.

COPY-READY PROMPT

```
Act as a senior change manager and Windows infrastructure engineer. Change: patch two domain controllers [versions/sites] in one approved maintenance window. Objective: maintain identity and DNS availability. Build a runbook with owners/placeholders, prechecks for replication, health, time, DNS, backups and recovery access; one-controller-at-a-time sequence; drain or dependency considerations; reboot and post-patch checks; authentication/DNS/replication validation; hold point before the second controller; measurable success and abort criteria; rollback/escalation path; user and stakeholder communications; and evidence required for closure. Challenge the plan before finalising it.
```

WHY THIS PROMPT WORKS

- Makes availability the operating objective
- Creates a hold point between systems
- Validates the service, not just the server
- Requires the plan to critique itself

CONDENSED SENIOR ENGINEER RESPONSE

Confirm replication, SYSVOL, DNS, time, monitoring, backups, and an independent recovery path before the window. Patch one controller, then validate local health plus real authentication, name resolution, and replication from dependent locations. Pause at a documented hold point and continue only if success thresholds remain satisfied for the agreed observation period. If the first controller does not recover cleanly or introduces replication risk, stop before touching the second, preserve evidence, and follow the approved rollback or escalation path.

NEXT-TURN REFINEMENT 1

```
Red-team this runbook: find the five most dangerous assumptions or ambiguous handoffs.
```

NEXT-TURN REFINEMENT 2

```
Convert the final runbook into an operator checklist and a separate stakeholder communication schedule.
```

WHAT THIS EXAMPLE DOES NOT PROVE

A complete-looking document cannot prove backup integrity, spare capacity, failover behaviour, access, timing, or owner availability. It also cannot decide that rollback is safer than continuing. Those claims require environment evidence, rehearsal where practical, and an authorised decision maker in the window.

TURN THE ANSWER INTO EVIDENCE

01	02	03	04	05
Confirm facts	**Run one safe check**	**Interpret result**	**Update the plan**	**Record the decision**

CHAPTER 12

Change Planning

Turn technical intent into a controlled, reviewable, and reversible runbook

PROMPT LIBRARY

Use these as starting structures. Replace every bracketed field, remove anything irrelevant, and keep unknowns explicit. The strongest template is the one that matches the real decision you need to make.

STANDARD CHANGE RUNBOOK

USE WHEN: a planned infrastructure or application change needs approval

```
Act as a senior change manager and [discipline] engineer. Objective: [outcome].
Scope/targets: [details]. Dependencies and business impact: [details].
Window/owners/approvals: [details]. Backup/recovery state: [evidence]. Produce
preparation, prechecks, ordered implementation, hold points, monitoring, service-level
validation, success/abort criteria, rollback with feasibility and duration,
communications, and closure evidence. Mark placeholders and assumptions. Finish with a risk
challenge.
```

EMERGENCY CHANGE BRIEF

USE WHEN: urgent remediation is necessary but control must remain

```
Prepare an emergency-change decision brief for [issue]. Current impact and urgency:
[facts]. Proposed action and scope: [details]. Evidence supporting it: [facts].
Alternatives considered: [details]. Time available: [details]. State authority required,
blast radius, prerequisites, backup/recovery, minimum safe test, monitoring, success/abort
criteria, rollback or compensating action, communications, and retrospective actions.
Distinguish urgent from unverified and do not invent approval.
```

CHANGE-PLAN CHALLENGE

USE WHEN: a runbook needs senior review

```
Review this runbook as a sceptical change advisory reviewer: [runbook]. Environment and
service criticality: [details]. Find missing dependencies, ambiguous commands, privilege
issues, sequence/race risks, capacity or failover assumptions, untested backups, weak
success/abort measures, unrealistic rollback, communication gaps, and ownership gaps. Rank
findings by risk and provide precise revisions. Do not approve the change; state what
evidence would make it review-ready.
```

BEFORE YOU SUBMIT

- ✓ Outcome and service impact are explicit
- ✓ Hold and abort points are measurable
- ✓ Closure requires evidence
- ✓ Every phase has an owner
- ✓ Rollback is feasible and timed

TEMPLATE USE SEQUENCE

01	02	03	04	05
Replace fields	**Remove irrelevant text**	**Mark unknowns**	**Check data handling**	**Verify the output**

CHAPTER 13

Security-Sensitive Support

Keep AI assistance defensive, evidence-aware, least-privileged, and policy controlled

DISCIPLINE CONTEXT

Security-sensitive prompts should focus on defensive outcomes: contain harm, preserve evidence, verify scope, recover safely, and communicate through the incident process. State the authority boundary and require escalation where policy, legal, privacy, or forensic decisions are involved.

Use data minimisation. Indicators, event summaries, and timestamps may be enough; raw mailbox contents, tokens, personal records, proprietary code, and complete forensic images usually are not. The approved case system remains the authoritative record, not the AI conversation.

PROMPT ENGINEERING LENS

WHY IT MATTERS

Ask for decision points and defensive checks, not evasion, exploitation, or destructive cleanup. Require the model to avoid disabling controls, deleting evidence, contacting an attacker, or making legal conclusions.

FIELD RULE

When security evidence and service recovery compete, surface the decision to the authorised incident lead.

THE DISCIPLINE SEQUENCE

01	02	03	04	05
Contain	**Preserve**	**Scope**	**Recover**	**Communicate**

INCLUDE IN THE PROMPT

- Asset/service criticality, symptoms, time, and severity
- Defensive objective and authority level
- Evidence location, retention, and chain-of-custody needs
- Security, privacy, legal, vendor, and communication escalation

AVOID THESE WEAK PATTERNS

- Uploading secrets or full sensitive datasets
- Destructive cleanup before evidence decisions
- Disabling security controls to test
- Treating model output as forensic or legal conclusion

MAKE THE REASONING VISIBLE

EVIDENCE	FALSIFY	CONTROL
What fact would most change the current ranking?	What result would disprove the leading cause?	Which next step is safe, reversible, and authorised?

OPERATIONAL CONTEXT

Keep the AI task narrow: organise defensive actions, evidence questions, scope checks, and communication triggers. Use approved case references rather than full sensitive artefacts. Record every containment or recovery decision in the incident system with the human approver, time, reason, expected impact, and validation outcome.

CHAPTER 13

2 / 3

Security-Sensitive Support

Keep AI assistance defensive, evidence-aware, least-privileged, and policy controlled

WORKED FIELD EXAMPLE

SCENARIO

A workstation begins making unusual outbound connections after a suspicious attachment is opened. Endpoint tooling is available, but the scope and severity are not yet known. Evidence must be preserved before cleanup.

COPY-READY PROMPT

```
Act as defensive security-operations support. Scenario: a managed workstation made unusual outbound connections after a suspicious attachment was opened at [time]. Current evidence: [sanitised EDR/network/email indicators]. Authority: propose actions only; incident lead approves containment and remediation. Build a defensive plan prioritising user safety, network containment through approved tooling, volatile and durable evidence preservation, endpoint/account/email scope checks, indicator search across the estate, severity and legal/privacy escalation triggers, recovery prerequisites, and communications. Label facts and hypotheses. Do not suggest destructive cleanup, credential collection, attacker interaction, or disabling controls.
```

WHY THIS PROMPT WORKS

- Defines a defensive purpose and approval boundary
- Preserves evidence before remediation
- Expands scope across related control planes
- Excludes high-risk and unauthorised actions

CONDENSED SENIOR ENGINEER RESPONSE

Notify the incident lead and use approved endpoint controls to isolate the device if the severity criteria are met, while preserving required telemetry and documenting time and actor. Protect the user account through the established identity-response process if evidence supports compromise. Search approved telemetry for the same indicators across endpoints, email, identity, and network sources. Do not delete files, reimage, or reset broadly until evidence, scope, recovery, and legal or privacy requirements are clear. Maintain an action log and define the evidence required to release the device back to service.

NEXT-TURN REFINEMENT 1

```
Turn these sanitised indicators into a defensive scope checklist without adding unsupported IOCs.
```

NEXT-TURN REFINEMENT 2

```
Review the containment plan for evidence loss, business impact, privacy, and approval gaps.
```

WHAT THIS EXAMPLE DOES NOT PROVE

An indicator is not proof of compromise, and the absence of one is not proof of safety. Isolation can protect the estate but may affect service and evidence. The model cannot determine forensic, privacy, legal, or notification obligations; those decisions belong to the authorised incident process.

TURN THE ANSWER INTO EVIDENCE

01	02	03	04	05
Confirm facts	**Run one safe check**	**Interpret result**	**Update the plan**	**Record the decision**

CHAPTER 13

3 / 3

Security-Sensitive Support

Keep AI assistance defensive, evidence-aware, least-privileged, and policy controlled

PROMPT LIBRARY

Use these as starting structures. Replace every bracketed field, remove anything irrelevant, and keep unknowns explicit. The strongest template is the one that matches the real decision you need to make.

SUSPICIOUS ENDPOINT RESPONSE

USE WHEN: an endpoint may be compromised

```
Act as defensive SOC support. Asset/user criticality: [minimised]. Trigger/time:
[details]. Sanitised EDR/network indicators: [summary]. Current containment state:
[details]. Authority and policy: [details]. Build a plan for approved containment, evidence
preservation, endpoint/account/network/email scoping, severity and escalation, recovery
prerequisites, monitoring, and documentation. Separate facts and hypotheses. No
destructive cleanup, control disabling, credential collection, exploitation, or attacker
interaction.
```

SUSPECTED ACCOUNT COMPROMISE

USE WHEN: identity activity may be unauthorised

```
Act as a defensive identity-response engineer. Account/resource: [sanitised]. Trigger and
time: [details]. Sign-in/audit/mailbox/app evidence: [sanitised]. Known user activity:
[confirmed/unknown]. Current controls: [details]. Propose policy-aligned containment
options for authorised approval, evidence preservation, session/credential/app-consent
review, scope across identities and resources, communication, recovery, and monitoring. Do
not request secrets or declare compromise without evidence.
```

PHISHING TRIAGE

USE WHEN: a suspicious message or link is reported

```
Act as defensive email-security support. Message/report context: [sanitised headers,
sender domain, delivery time, user action]. Security-tool verdicts: [summary]. Scope
evidence: [recipient/search summary]. Build a safe triage covering message authentication
and routing evidence, attachment/link detonation results already available, recipient and
click scope, containment through approved controls, account/endpoint follow-up,
preservation, user communication, and escalation. Do not open unknown content or invent
indicators.
```

BEFORE YOU SUBMIT

- ✓ The objective is explicitly defensive
- ✓ Data is minimised and sanitised
- ✓ Recovery has release criteria
- ✓ Authority and escalation are named
- ✓ Evidence precedes destructive action

TEMPLATE USE SEQUENCE

01	02	03	04	05
Replace fields	**Remove irrelevant text**	**Mark unknowns**	**Check data handling**	**Verify the output**

CHAPTER 14

Communication and Documentation

Use one verified fact pack to create the right message for each audience

DISCIPLINE CONTEXT

Technical work produces different documents from the same event: an executive update, user notice, engineering handoff, ticket, change record, and post-incident review. Prompting improves consistency when the model is given one authoritative fact pack and a separate audience, purpose, tone, and length for each output.

Require facts, decisions, actions, owners, and uncertainty to remain consistent. Executives need impact and decision confidence; users need practical effect and the next update; engineers need evidence and exact actions. Never let the model invent an ETA, root cause, owner, or completed action.

PROMPT ENGINEERING LENS

WHY IT MATTERS

Ask the model to transform, not embellish. Provide a terminology list, confidentiality level, required fields, and words to avoid. For live incidents, refresh the fact pack and show the as-of time on every message.

FIELD RULE

Change the level of detail, not the underlying facts.

THE DISCIPLINE SEQUENCE

01	02	03	04	05
Facts	**Audience**	**Purpose**	**Format**	**Verify**

INCLUDE IN THE PROMPT

- Authoritative facts and as-of timestamp
- Audience, decision or action needed, and channel
- Tone, length, terminology, confidentiality, and owner
- Known unknowns, next update time, and review requirement

AVOID THESE WEAK PATTERNS

- One message for every audience
- Unsupported reassurance or technical certainty
- Invented root cause or restoration time
- Copying sensitive technical detail into broad channels

MAKE THE REASONING VISIBLE

EVIDENCE	FALSIFY	CONTROL
What fact would most change the current ranking?	What result would disprove the leading cause?	Which next step is safe, reversible, and authorised?

OPERATIONAL CONTEXT

Create and freeze a verified fact pack before generating multiple messages. Give each output an audience, channel, owner, maximum length, confidentiality level, and as-of time. Review the drafts side by side so impact, scope, actions, confidence, and the next-update commitment remain consistent while technical depth changes.

CHAPTER 14

Communication and Documentation

Use one verified fact pack to create the right message for each audience

WORKED FIELD EXAMPLE

SCENARIO

A service incident requires four outputs: an executive update, an end-user notice, a technical handoff, and a ticket summary. The root cause is not confirmed and the next checkpoint is in 30 minutes.

COPY-READY PROMPT

```
Act as a senior technical communicator. Use only this authoritative fact pack: [timestamped facts]. Root cause status: unconfirmed. Next checkpoint: [time/timezone]. Create four separate outputs: (1) executive update led by business impact, confidence, decision needs, and next checkpoint; (2) end-user notice with service effect, any safe user action, and next update; (3) engineering handoff with scope, timeline, evidence, actions/results, hypotheses, owners, blockers, and next tests; (4) concise ticket summary. Keep facts and times consistent, mark unknowns, preserve confidentiality, and do not invent an ETA or completed action.
```

WHY THIS PROMPT WORKS

- Uses a single authoritative source
- Defines four different reader needs
- Makes uncertainty visible
- Blocks the most common incident-writing inventions

CONDENSED SENIOR ENGINEER RESPONSE

The executive version should state impact, trend, confidence, and decisions due without low-level logs. The user version should be plain, practical, and time-bound to the next update. The handoff should preserve the full technical chain and clearly separate evidence from hypotheses. The ticket should capture searchable facts, actions, and ownership. All four must use the same as-of time and avoid any restoration estimate or root-cause statement not present in the fact pack.

NEXT-TURN REFINEMENT 1

```
Compare the four drafts and flag any factual, timing, ownership, or confidence mismatch.
```

NEXT-TURN REFINEMENT 2

```
Rewrite the end-user notice at plain-English reading level without losing the next-update commitment.
```

WHAT THIS EXAMPLE DOES NOT PROVE

A fact pack can be incomplete or stale, and a well-written transformation can preserve the same underlying error across every audience. Plain language can also remove important conditions if shortened carelessly. A named owner must verify the facts and approve publication in the correct channel.

TURN THE ANSWER INTO EVIDENCE

01	02	03	04	05
Confirm facts	**Run one safe check**	**Interpret result**	**Update the plan**	**Record the decision**

CHAPTER 14

Communication and Documentation

Use one verified fact pack to create the right message for each audience

PROMPT LIBRARY

Use these as starting structures. Replace every bracketed field, remove anything irrelevant, and keep unknowns explicit. The strongest template is the one that matches the real decision you need to make.

EXECUTIVE SERVICE UPDATE

USE WHEN: leaders need impact and decisions, not raw diagnostics

```
Using only this verified fact pack [facts], write an executive service update as of
[time/timezone]. Include business impact, affected scope, trend, actions under way,
current confidence, risks, decisions or support needed, and next update time. Maximum
[length]. Avoid technical detail unless it changes a decision. Mark root cause and ETA as
unknown unless explicitly confirmed.
```

END-USER NOTICE

USE WHEN: people need clear service guidance

```
Using only these verified facts [facts], write a plain-language user notice for
[channel/audience]. State what is affected, who may notice it, any safe workaround or user
action that is confirmed, what the support team is doing at a high level, and the next
update time. Keep to [length/tone]. Do not blame, speculate, expose sensitive detail, or
promise a restoration time not in the facts.
```

TECHNICAL HANDOFF OR TICKET

USE WHEN: another engineer must continue without re-discovery

```
Create a technical handoff from this timestamped fact pack [facts]. Include
environment/service, impact and scope, timeline in one timezone, exact symptoms/errors,
healthy comparisons, evidence collected, actions attempted with results,
confirmed/rejected hypotheses, current ranking, owners, blockers, access/change
constraints, and the next two tests. Keep commands and values exact where supplied; do not
fill missing data.
```

BEFORE YOU SUBMIT

- ✓ The fact pack has an as-of time
- ✓ Unknowns remain unknown
- ✓ The next update or owner is clear
- ✓ Audience and action are explicit
- ✓ No sensitive detail crosses audience boundaries

TEMPLATE USE SEQUENCE

01	02	03	04	05
Replace fields	**Remove irrelevant text**	**Mark unknowns**	**Check data handling**	**Verify the output**

CHAPTER 15

Reusable Prompt Library

Turn good individual prompts into a maintained, testable team capability

DISCIPLINE CONTEXT

A useful prompt library is small enough to use and structured enough to trust. Store templates by task, not by fashionable model. Each should have a purpose, owner, version, approved data classification, required fields, expected output, example input, and review date.

Templates should create better questions without forcing every incident into the same shape. Make unknowns valid inputs, include a healthy comparison where relevant, and add an instruction for the model to request missing evidence. Test templates against realistic good, incomplete, and misleading cases before publishing them.

PROMPT ENGINEERING LENS

WHY IT MATTERS

Treat prompt templates like operational documentation. Review them when tools, policies, environments, or model behaviour change. Capture what a reviewer must verify rather than assuming a polished output is correct.

FIELD RULE

A reusable prompt should improve the question, constrain the answer, and make review easier.

THE DISCIPLINE SEQUENCE

01	02	03	04	05
Purpose	**Fields**	**Guardrails**	**Test cases**	**Version**

INCLUDE IN THE PROMPT

- Task, audience, owner, version, and review date
- Required and optional placeholders
- Data-handling guardrails and prohibited inputs
- Expected response schema, example, and validation checklist

AVOID THESE WEAK PATTERNS

- Hundreds of near-duplicate prompts
- Templates tied to one incident's values
- No owner or retirement process
- Measuring quality by eloquence alone

MAKE THE REASONING VISIBLE

EVIDENCE	FALSIFY	CONTROL
What fact would most change the current ranking?	What result would disprove the leading cause?	Which next step is safe, reversible, and authorised?

OPERATIONAL CONTEXT

Publish a small curated set where engineers already work, with examples and a clear route for feedback. Track template version separately from model version and rerun test cases after material changes. Usage counts may show adoption, but quality review should focus on evidence handling, safety, decision usefulness, and the amount of rework required.

CHAPTER 15

Reusable Prompt Library

Turn good individual prompts into a maintained, testable team capability

WORKED FIELD EXAMPLE

SCENARIO

A team repeatedly asks AI to troubleshoot incidents, draft changes, and review scripts. Existing prompts are stored in personal notes, use inconsistent fields, and do not include data-handling or validation instructions.

COPY-READY PROMPT

```
Act as a prompt-library curator for an IT operations team. We need three governed
templates: troubleshooting, change planning, and code review. For each define purpose,
owner placeholder, permitted data classification, prohibited inputs, required fields,
optional fields, copy-ready prompt, expected response headings, operator validation
checklist, one complete example, one incomplete-input test, one misleading-evidence test,
version, and review trigger. Keep shared fields consistent but preserve task-specific
logic. Flag any instruction that could encourage secrets, unsupported certainty,
unapproved change, or over-reliance on self-critique.
```

WHY THIS PROMPT WORKS

- Treats prompts as maintained operational assets
- Builds data handling into the template
- Tests failure cases, not only ideal inputs
- Keeps shared structure without flattening disciplines

CONDENSED SENIOR ENGINEER RESPONSE

Create a common header for environment, scope, evidence, recent changes, constraints, and desired output, then add task-specific sections. Troubleshooting needs hypotheses and discriminating tests; change planning needs authority, success, abort, and rollback; code review needs runtime, privilege, side effects, security, and tests. Each template should reject secrets, label unknowns, and include an operator review checklist. Version changes should record why the prompt changed and which test cases were rerun.

NEXT-TURN REFINEMENT 1

```
Score these templates against clarity,
evidence use, safety, response usefulness,
and reviewability.
```

NEXT-TURN REFINEMENT 2

```
Consolidate this prompt set, preserving
genuinely different workflows and retiring
duplicates with reasons.
```

WHAT THIS EXAMPLE DOES NOT PROVE

No template can cover every environment, model update, or adversarial input. A passing test set may miss a new failure mode, and over-standardisation can hide an unusual incident. Templates should guide framing while leaving the operator free to remove, add, or reject fields that do not fit.

TURN THE ANSWER INTO EVIDENCE

01	02	03	04	05
Confirm facts	**Run one safe check**	**Interpret result**	**Update the plan**	**Record the decision**

CHAPTER 15

Reusable Prompt Library

Turn good individual prompts into a maintained, testable team capability

PROMPT LIBRARY

Use these as starting structures. Replace every bracketed field, remove anything irrelevant, and keep unknowns explicit. The strongest template is the one that matches the real decision you need to make.

UNIVERSAL TROUBLESHOOTING

USE WHEN: a team needs one dependable diagnostic starting point

```
Act as my senior [discipline] engineer. Environment/service: [details]. Business purpose/criticality: [details]. Symptom and onset: [details]. Affected and unaffected scope: [details]. Last known good: [details]. Evidence with timestamps: [details]. Recent changes: [details/none known]. Attempts and results: [details]. Constraints/authority: [details]. Ask up to five high-value questions if needed; then rank hypotheses, give read-only discriminating tests with result branches, define validation, stop/escalation conditions, and label facts, assumptions, and unknowns.
```

UNIVERSAL CHANGE PLAN

USE WHEN: a technical intention must become a controlled runbook

```
Act as a senior change manager and [discipline] engineer. Objective and reason: [details]. Scope/targets/dependencies: [details]. Risk and business impact: [details]. Window/owners/approvals: [details]. Backup/recovery evidence: [details]. Produce preparation, prechecks, ordered steps, hold points, monitoring, service-level success and abort criteria, rollback feasibility and duration, communications, and closure evidence. Challenge the plan for hidden assumptions and mark all placeholders.
```

UNIVERSAL TECHNICAL REVIEW

USE WHEN: an answer, script, runbook, or diagnosis needs challenge

```
Review this artefact as a sceptical senior [discipline] engineer: [artefact]. Intended environment, purpose, authority, and constraints: [details]. Separate facts, assumptions, and recommendations. Find correctness issues, unsafe actions, security/privacy concerns, hidden dependencies, blast radius, missing validation, weak rollback, unsupported certainty, and operator ambiguity. Rank findings, explain evidence, provide the smallest safe revision, and list independent checks still required.
```

BEFORE YOU SUBMIT

- ✓ Every template has an owner and version
- ✓ Data rules are embedded
- ✓ Good and adversarial test cases exist
- ✓ Required fields match the task
- ✓ Expected output is reviewable

TEMPLATE USE SEQUENCE

01	02	03	04	05
Replace fields	**Remove irrelevant text**	**Mark unknowns**	**Check data handling**	**Verify the output**

CHAPTER 16

Review Before You Act

Treat model output as untrusted technical advice until evidence and authority support it

DISCIPLINE CONTEXT

AI can produce a confident answer from incomplete or incorrect premises. Review it like advice from a capable colleague who cannot see your environment. Confirm commands, versions, assumptions, dependencies, privileges, data handling, blast radius, reversibility, validation, rollback, and change authority.

Self-critique is useful but not independent assurance. A model may repeat the same mistaken premise in a more polished form. Compare recommendations with primary documentation, live read-only evidence, lab testing, code review, monitoring, and an authorised human decision when risk warrants it.

PROMPT ENGINEERING LENS

WHY IT MATTERS

Ask the model to expose uncertainty and attack its own answer, then verify the result outside the conversation. High impact, irreversible, security-sensitive, or poorly understood actions require a stricter review and often a peer.

FIELD RULE

No model response outranks verified environment evidence, policy, or authorised engineering judgement.

THE DISCIPLINE SEQUENCE

01	02	03	04	05
Assumptions	**Evidence**	**Safety**	**Rollback**	**Decision**

INCLUDE IN THE PROMPT

- Confirmed facts and source of truth
- Environment/version compatibility and prerequisites
- Privilege, side effects, blast radius, and data risk
- Success, abort, rollback, monitoring, approval, and owner

AVOID THESE WEAK PATTERNS

- Equating detail with correctness
- Running commands you cannot explain
- Using AI self-review as the only review
- Proceeding because rollback sounds easy

MAKE THE REASONING VISIBLE

EVIDENCE	FALSIFY	CONTROL
What fact would most change the current ranking?	What result would disprove the leading cause?	Which next step is safe, reversible, and authorised?

OPERATIONAL CONTEXT

Match the review depth to impact and reversibility. A read-only query on one test target may need syntax and access checks; a production identity, security, network, or data change may need primary documentation, peer review, lab evidence, formal approval, monitoring, and rehearsed recovery. Record the residual risk the human owner accepted.

CHAPTER 16

Review Before You Act

Treat model output as untrusted technical advice until evidence and authority support it

WORKED FIELD EXAMPLE

SCENARIO

An AI answer recommends a service restart and configuration edit to resolve intermittent production access failures. It does not show which evidence identifies the service, how the edit was validated, or what happens if the restart expands the outage.

COPY-READY PROMPT

```
Review your previous recommendation as a sceptical senior change reviewer. Environment and constraints: [details]. Proposed restart/configuration edit: [paste]. First separate confirmed facts, assumptions, and recommendations. Then identify unsupported causal claims, version or syntax uncertainty, prerequisites, read/write effects, privilege, security and data impact, blast radius, service dependencies, validation gaps, monitoring, abort conditions, rollback feasibility, approval needs, and safer read-only checks that should come first. Provide a revised answer that clearly states what evidence would justify the change. Do not treat your self-review as final approval.
```

WHY THIS PROMPT WORKS

- Targets the exact unsupported leap
- Reviews technical and operational risk
- Moves read-only evidence ahead of change
- Keeps approval outside the model

CONDENSED SENIOR ENGINEER RESPONSE

The restart and edit are not justified until evidence links the failure to that service and configuration. Confirm the symptom at a timestamp, compare healthy and failing paths, validate the configuration syntax and version, and identify dependencies and current state. Define a service-level success measure, monitoring window, abort threshold, and a tested restoration method for the original configuration. If the failure remains intermittent or the change has broad impact, reproduce in a safe environment or obtain peer review before the production window.

NEXT-TURN REFINEMENT 1

Show the strongest evidence against your own leading recommendation and explain what would change your decision.

NEXT-TURN REFINEMENT 2

Convert the final recommendation into a go/no-go decision record with owner, authority, evidence, and residual risk.

WHAT THIS EXAMPLE DOES NOT PROVE

Self-critique is generated by the same model family and may preserve the same hidden premise. It is a useful way to expose questions, not an independent control. Evidence outside the conversation, including live state, documentation, tests, policy, and accountable review, remains necessary.

TURN THE ANSWER INTO EVIDENCE

01	02	03	04	05
Confirm facts	**Run one safe check**	**Interpret result**	**Update the plan**	**Record the decision**

CHAPTER 16

3 / 3

Review Before You Act

Treat model output as untrusted technical advice until evidence and authority support it

PROMPT LIBRARY

Use these as starting structures. Replace every bracketed field, remove anything irrelevant, and keep unknowns explicit. The strongest template is the one that matches the real decision you need to make.

ASSUMPTION AND EVIDENCE AUDIT

USE WHEN: an answer sounds plausible but may rest on weak premises

```
Audit this technical answer: [answer]. Original facts: [facts]. Create three lists:
confirmed facts, inferences, and unsupported assumptions. For every recommendation, map
the evidence that supports it, evidence that contradicts it, and the missing observation
that would most change the decision. Flag version-specific claims or commands requiring
primary-source confirmation. Rewrite only after the evidence gaps are visible.
```

SAFETY AND CHANGE REVIEW

USE WHEN: an answer contains commands or configuration changes

```
Review these proposed actions as a senior production-safety reviewer: [actions].
Environment/service criticality: [details]. For each action state read/write effect,
privilege, targets, blast radius, dependencies, security/privacy impact, reversibility,
backup requirement, test method, expected result, failure signal, abort threshold,
rollback/compensation, monitoring, owner, and approval. Move safer evidence-gathering
ahead of change and reject any step that cannot be explained or bounded.
```

GO/NO-GO DECISION RECORD

USE WHEN: a human owner must decide whether to proceed

```
Using only these verified facts and review findings [details], draft a go/no-go decision
record. Include objective, options, confirmed evidence, unresolved uncertainty, risk and
blast radius, prerequisites, authority/owner, success and abort measures, rollback
evidence, monitoring, residual risk, and the explicit decision required. Do not make the
decision or invent approval. State what additional evidence would convert a no-go or hold
into a defensible go.
```

BEFORE YOU SUBMIT

- ✓ Commands and claims are independently verified
- ✓ Read/write effects and blast radius are known
- ✓ Success, abort, and rollback are credible
- ✓ Authority and owner are explicit
- ✓ Residual uncertainty is visible

TEMPLATE USE SEQUENCE

01	02	03	04	05
Replace fields	**Remove irrelevant text**	**Mark unknowns**	**Check data handling**	**Verify the output**